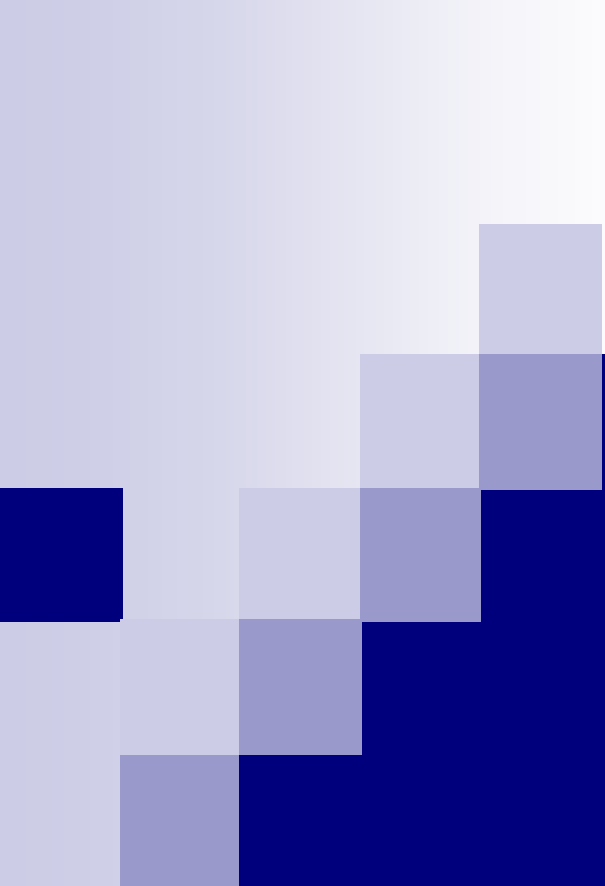




Computação Eletrônica

Introdução a C e Variáveis Inteiras

C

- Criado pelo suíço Dennis Ritchie no início da década 70 no laboratório da AT&T Bell Labs
- O nome está associado à evolução da linguagem de programação "B"
- Uma das linguagens de programação mais utilizada de todos os tempos



Dennis Ritchie



printf

Hello World

- Um programa que escreve "Hello World" no monitor

```
#include <stdio.h>
int main() {

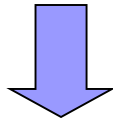
    printf("Hello World");

    return 0;
}
```

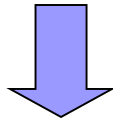
Compilação

```
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

helloWorld.c

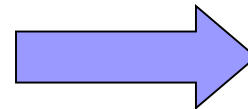


Compilador C (GNU, Code
Blocks, etc)

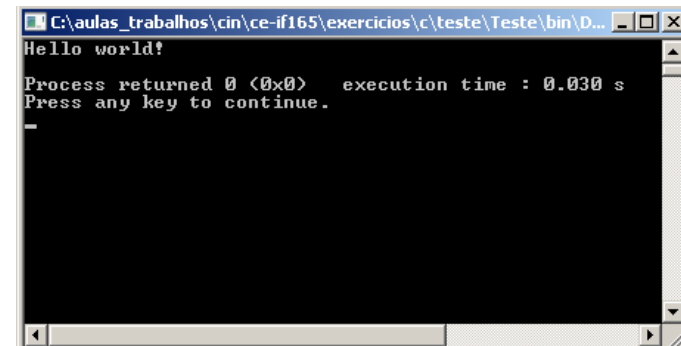


```
101010011101010101010101010101010
01000101010101010101011101110101
010101010101010101010101010101010
```

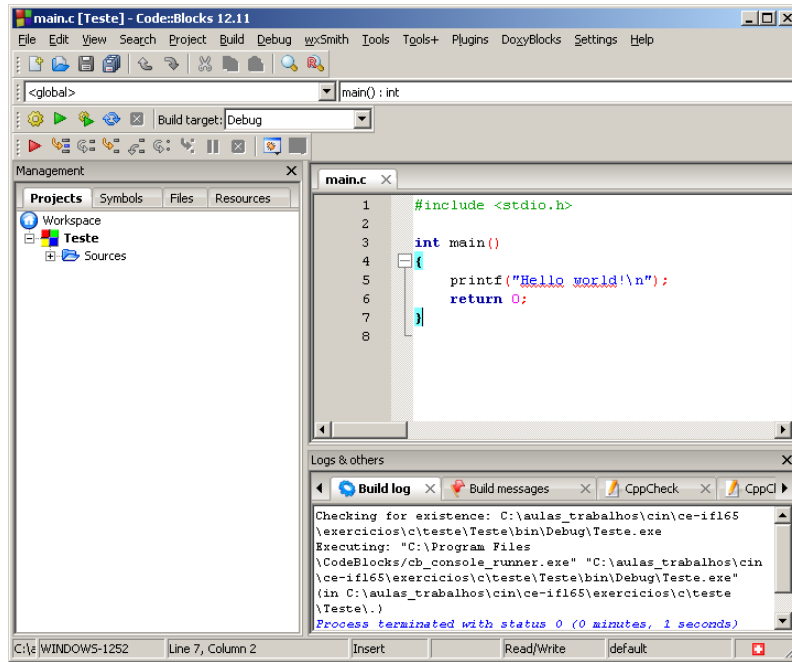
helloWorld.exe



duplo clique

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\aulas_trabalhos\cin\ce-if165\exercicios\c\teste\Teste\bin\D...'. The window content displays 'Hello world!' followed by 'Process returned 0 (0x0) execution time : 0.030 s' and 'Press any key to continue.' on the next line. The cursor is positioned at the start of the second line.

Compilação



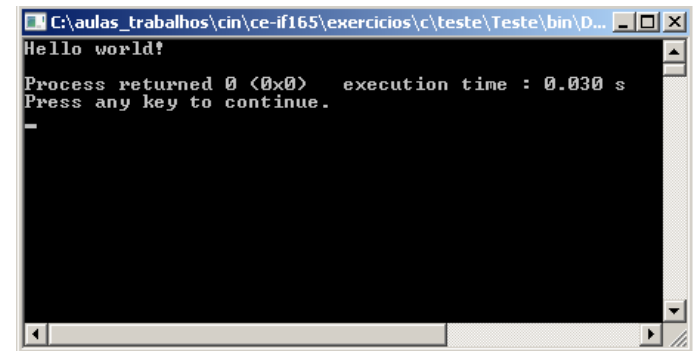
CodeBlocks grava o arquivo
bin\Debug\helloWorld.exe na
mesma pasta de helloWorld.c

```
Checking for existence: C:\aulas_trabalhos\cin\ce-ifl65
\exercicios\c\teste\Teste\bin\Debug\Teste.exe
Executing: "C:\Program Files
\CodeBlocks\cb_console_runner.exe" "C:\aulas_trabalhos\cin
\ce-ifl65\exercicios\c\teste\Teste\bin\Debug\Teste.exe"
(in C:\aulas_trabalhos\cin\ce-ifl65\exercicios\c\teste
\Teste\.)
Process terminated with status 0 (0 minutes, 1 seconds)
```

Build -> Run

Build-> Build

CodeBlocks com o
arquivo helloWorld.c



Compilação

■ Observação

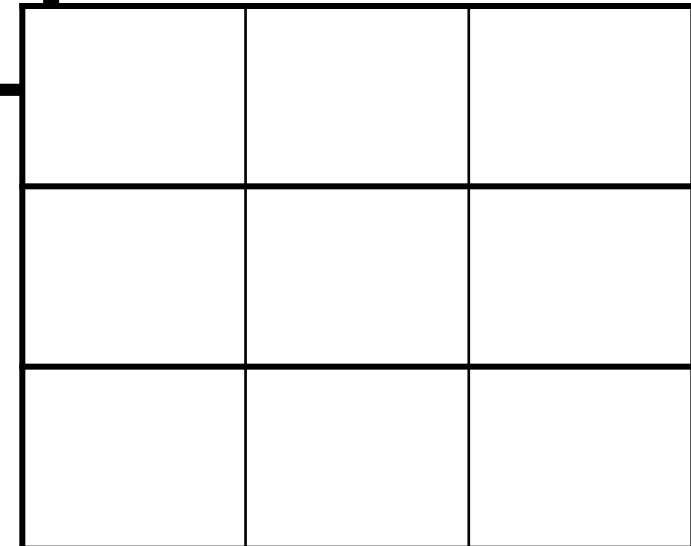
- ☐ Os programas construídos neste curso serão exclusivamente modo texto.
- ☐ Ou seja, não haverá mouse, janela, ícones ou gráficos. Apenas texto.
- ☐ A execução será feita através de entradas de dados pelo teclado e saídas de textos no monitor (dentro de uma janela Command Prompt)

Execução

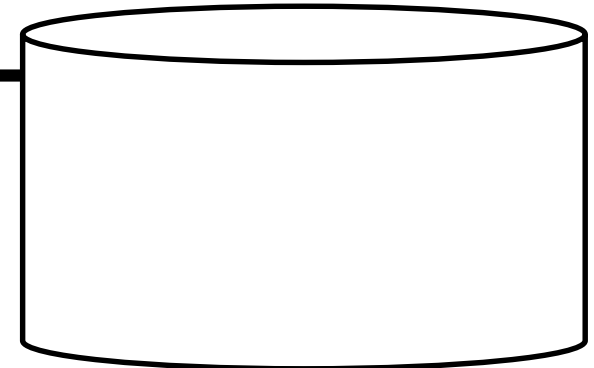
10101001110101010101010101010101001000101010101010101
1011101
01010101010101010101101010101010101010101010101010101

*O que acontece após o
duplo clique em
helloWorld.exe?*

Memória



HD



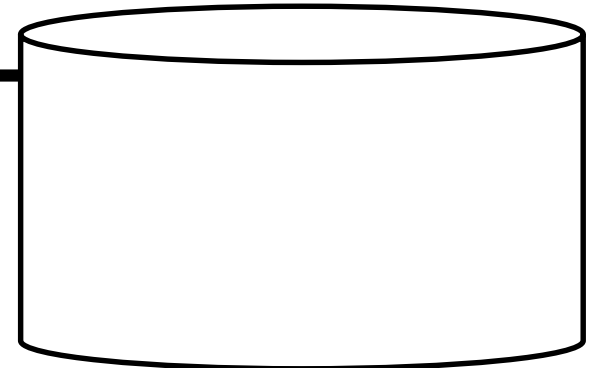
Execução

1010100111010101010101010101010010001010101010101
1011101
0101010101010101011010101010101010101010101010101

Hello World

Memória

HD

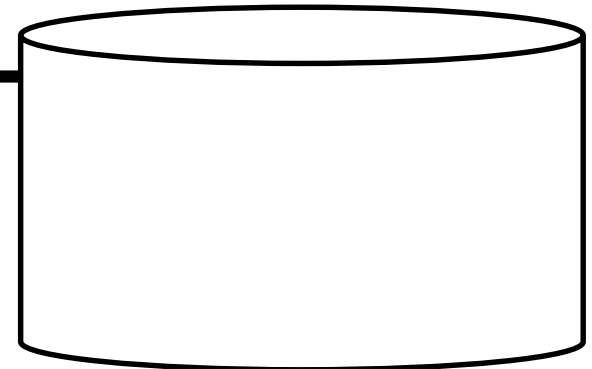


Execução

1010100111010101010101010101010010001010101010101
1011101
0101010101010101010110101010101010101010101010101

Memória

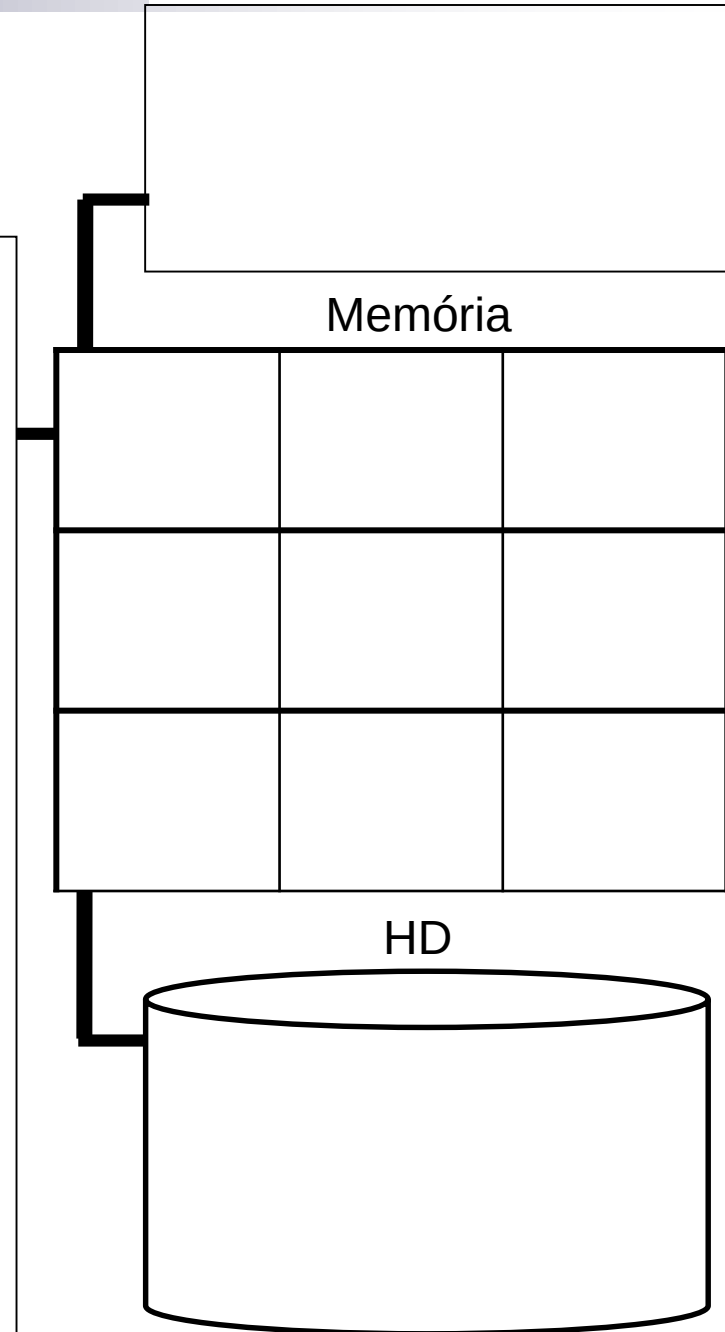
HD



Execução

```
#include <stdio.h>
→ int main() {
    printf("Hello World");
    return 0;
}
```

Para facilitar o entendimento, mostraremos o código fonte ao invés dos zeros e uns

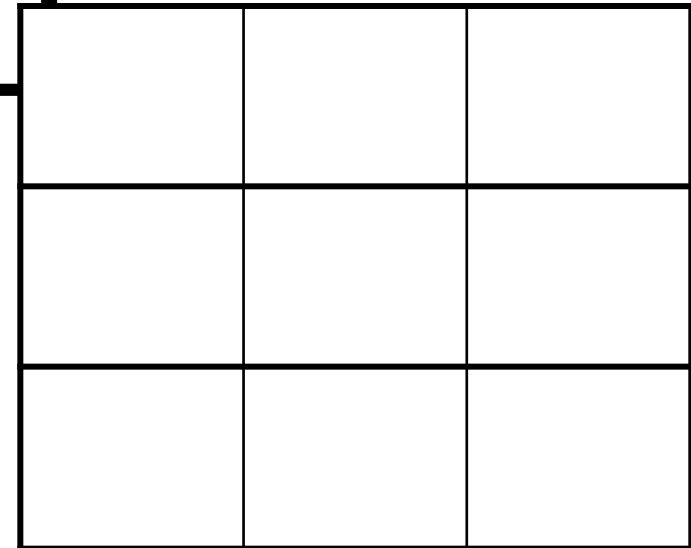


Execução

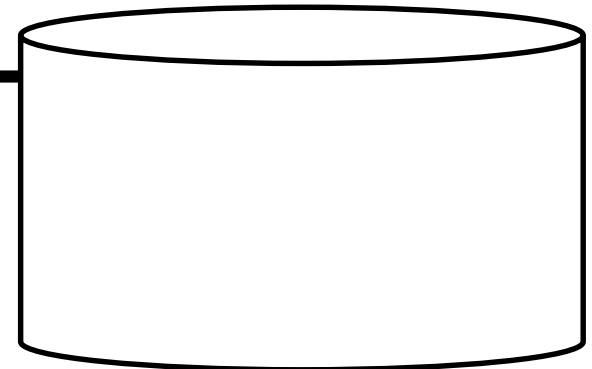
```
#include <stdio.h>
int main() {
    → printf("Hello World");
    return 0;
}
```

Hello World

Memória

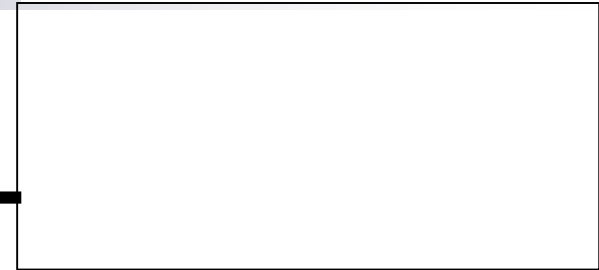


HD

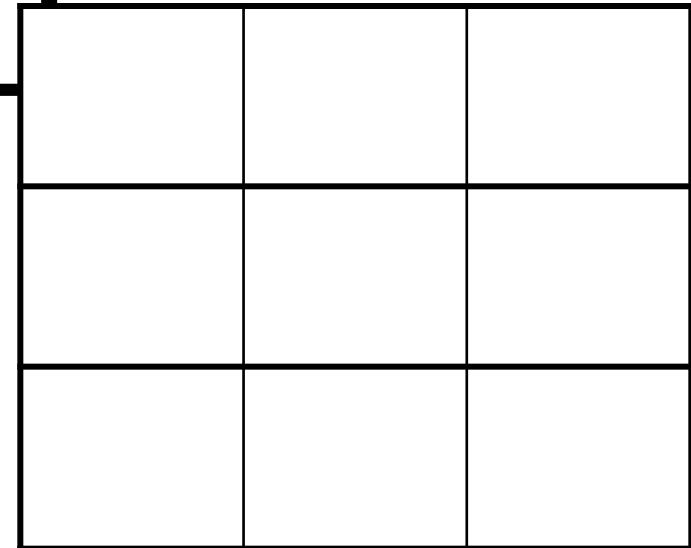


Execução

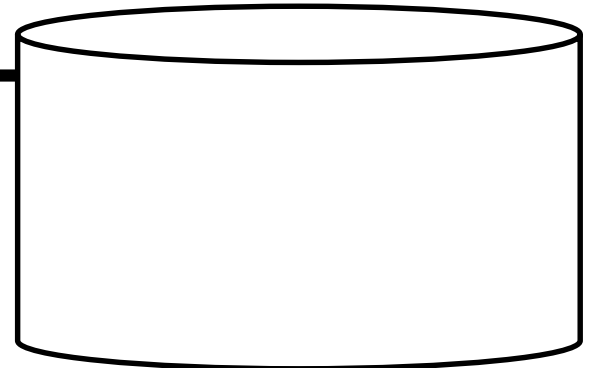
```
#include <stdio.h>
int main() {
    printf("Hello World");
    → return 0;
}
```



Memória



HD



printf

- O que este programa faz?

```
#include <stdio.h>
int main() {

    printf("Hello World");
    printf("Alo Mundo");

    return 0;
}
```

printf

■ Conheça o \n

```
#include <stdio.h>
int main() {

    printf("Hello World \n");
    printf("Alo Mundo");

    return 0;
}
```

printf

- E agora? Muda alguma coisa?

```
#include <stdio.h>
int main() {

    printf("Hello World");
    printf("\nAlo Mundo");

    return 0;
}
```


printf

■ Tente estes!

```
#include <stdio.h>
int main() {

    printf("Hello World");
    printf("\n");
    printf("Alo Mundo");

    return 0;
}
```

```
#include <stdio.h>
int main() {

    printf("\n\n\nHello\nWorld\n\nAlo\nMundo");

    return 0;
}
```



scanf

scanf

- Um programa que aguarda o usuário digitar alguma letra ou número no teclado
- E finaliza assim que for teclado ENTER

```
#include <stdio.h>
int main() {

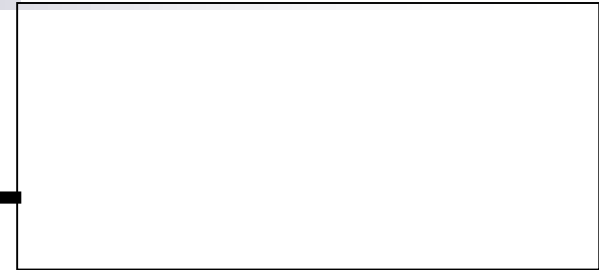
    scanf(" ");

    return 0;
}
```

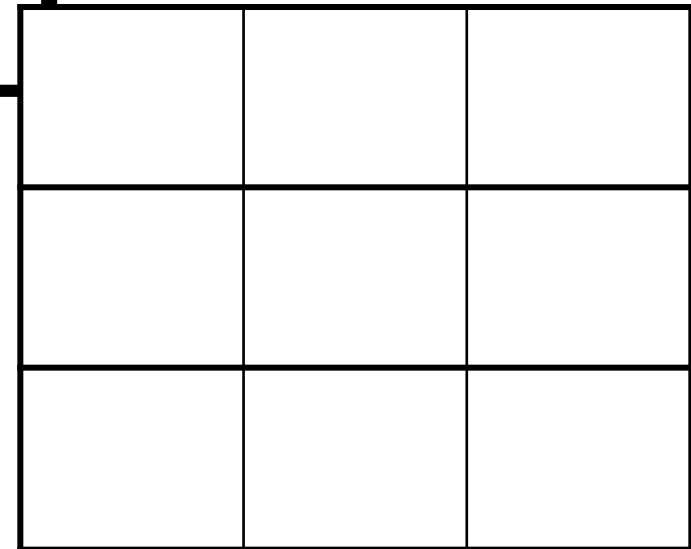
leTeclado.c

Execução

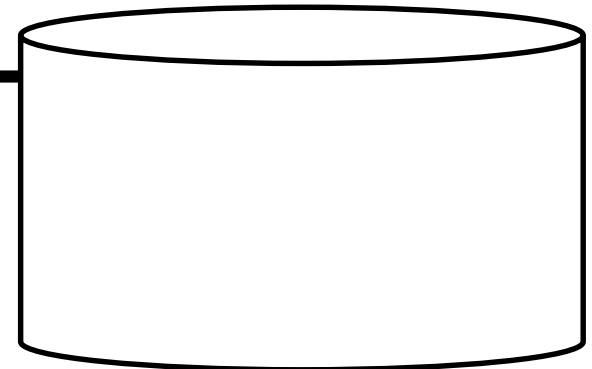
```
#include <stdio.h>
→ int main() {
    scanf(" ");
    return 0;
}
```



Memória



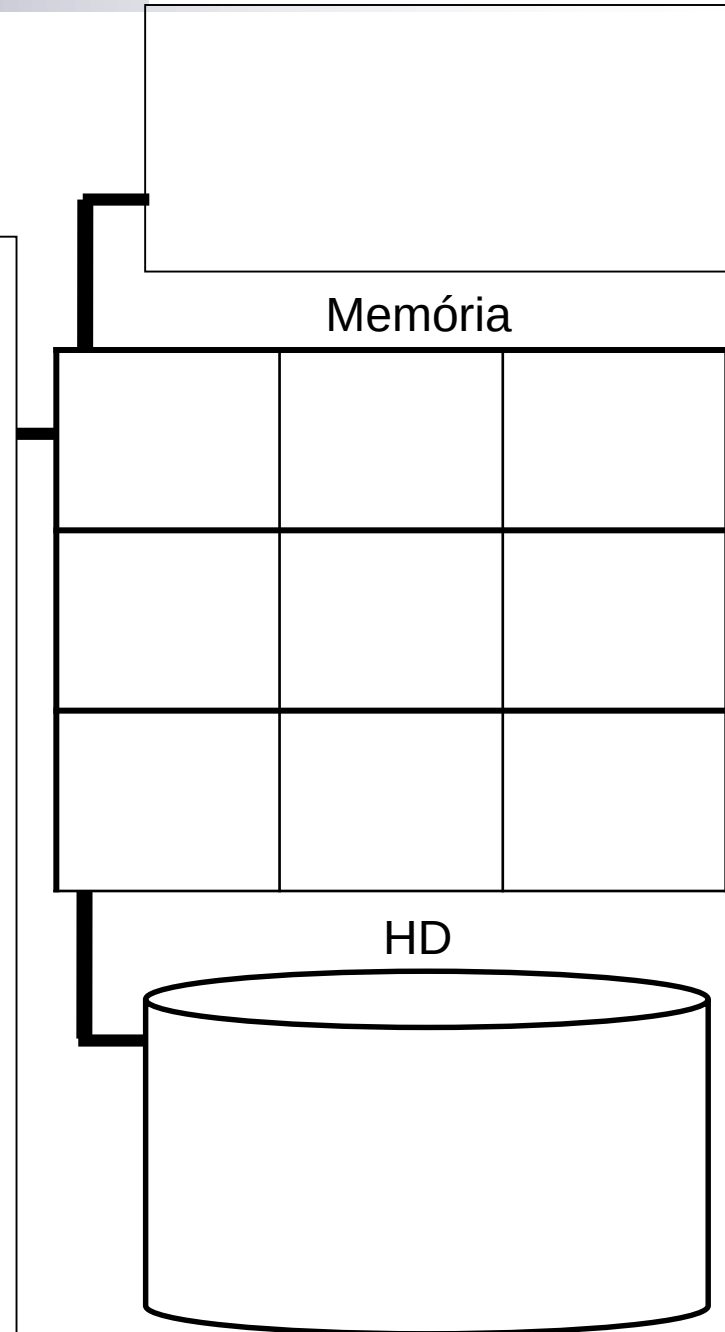
HD



Execução

```
#include <stdio.h>
int main() {
    → scanf(" ");
    return 0;
}
```

O computador coloca um cursor na tela indicando que espera uma digitação.



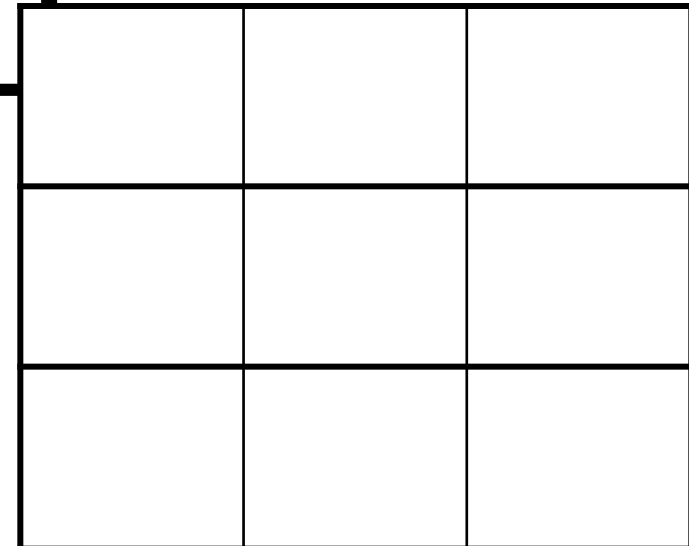
Execução

```
#include <stdio.h>
int main() {
    → scanf(" ");
    return 0;
}
```

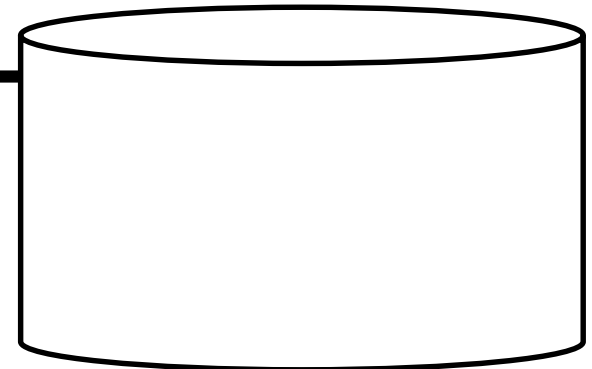
Vamos assumir que o usuário digitou 9. O scanf() continua aguardando mais digitação ou a finalização com um <ENTER>.

9

Memória



HD



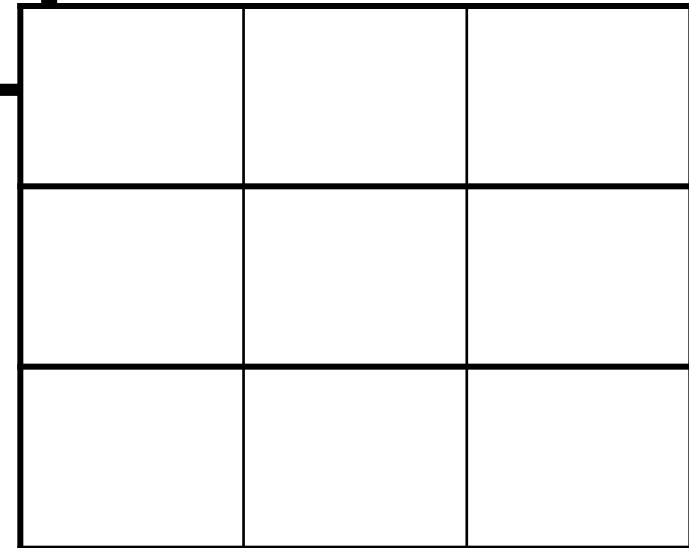
Execução

```
#include <stdio.h>
int main() {
    → scanf(" ");
    return 0;
}
```

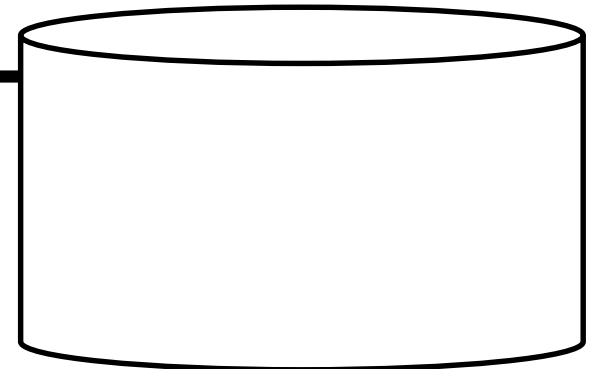
Vamos assumir que o usuário digitou <ENTER>.

9 <ENTER>

Memória

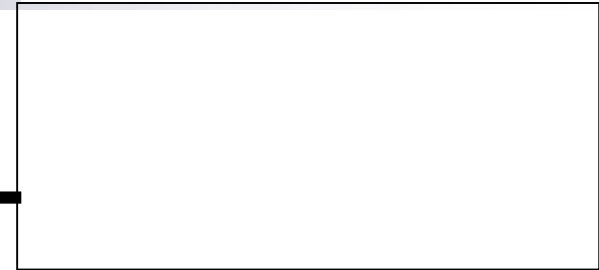


HD

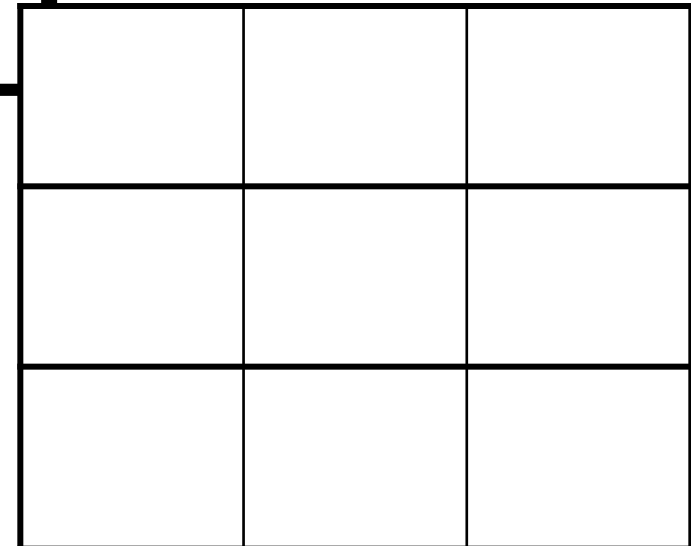


Execução

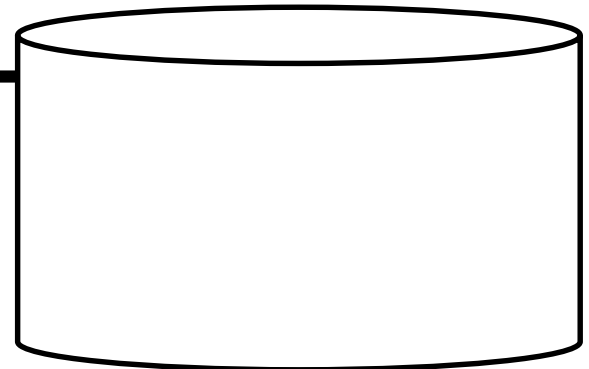
```
#include <stdio.h>
int main() {
    scanf(" ");
    → return 0;
}
```



Memória



HD



scanf

- O que o programa abaixo faz?

```
#include <stdio.h>
int main() {

    printf("Pressione algo e depois <ENTER> para sair.");
    scanf(" ");

    return 0;
}
```

scanf

■ E agora?

```
#include <stdio.h>
int main() {

    printf("Pressione algo e depois <ENTER> para sair.\n");
    scanf(" ");
    printf("Tchau");

    return 0;
}
```

scanf

■ E este? Faz diferença?

```
#include <stdio.h>
int main() {

    scanf(" ");
    printf("Pressione algo e depois <ENTER> para sair.\n");
    printf("Tchau");

    return 0;
}
```



A Memória

A Memória



- A memória pode ser vista como um imenso gaveteiro
- Em cada gaveta, guardamos uma informação. Por exemplo, um pedaço de papel contendo um número. Ou um pedaço de papel contendo um nome.



Variáveis

- As gavetas em programação são chamadas de **variáveis**
- Antes de utilizarmos uma variável, devemos nomeá-la.

Variáveis

- Em C os nomes de variáveis seguem a seguinte regra:
 - É formado por uma única letra ou por uma letra seguida de letras ou dígitos
 - Não é permitido espaço em branco ou outros caracteres como @ * , ; / :
 - \$ e _ podem ser utilizados
 - Exemplos permitidos: a, nota1, matricula, lucroTotal, _teste
 - Exemplos errados: 5B, A:B, km/h

Variáveis

- As palavras abaixo não podem ser utilizadas para nomear variáveis. Elas já possuem outro significado ou propósito. São denominadas palavras reservadas.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Variáveis

- Em C, devemos nomear a gaveta através da **declaração de uma variável**

```
#include <stdio.h>
```

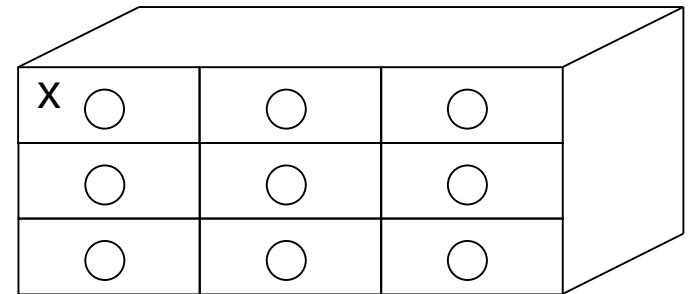
```
int main() {
```

```
    var x;
```

```
    printf("Hello World");
```

```
    return 0;
```

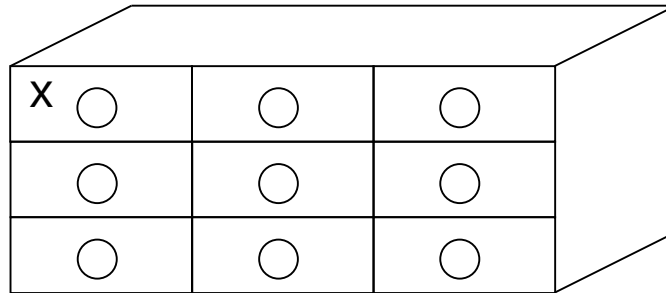
```
}
```



- Porém, este programa não compila!

Variáveis

- Computadores trabalham com variáveis de diferentes **tipos**:
 - Em C: int, double, float, char
 - Vamos trabalhar apenas com inteiros (int) por enquanto (tipicamente, -2.147.483.648 ... 2.147.483.647)

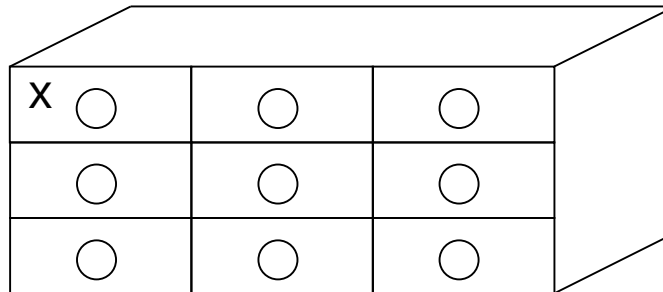


Variáveis

- O programa anterior deve ser corrigido para:

```
#include <stdio.h>

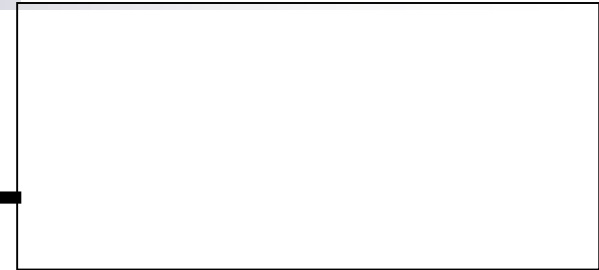
int main() {
    int x;
    printf("Hello World");
    return 0;
}
```



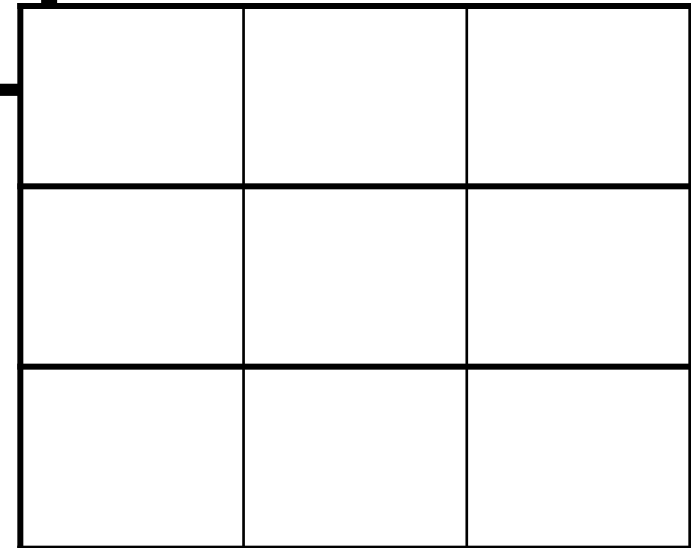
Execução

```
#include<stdio.h>
```

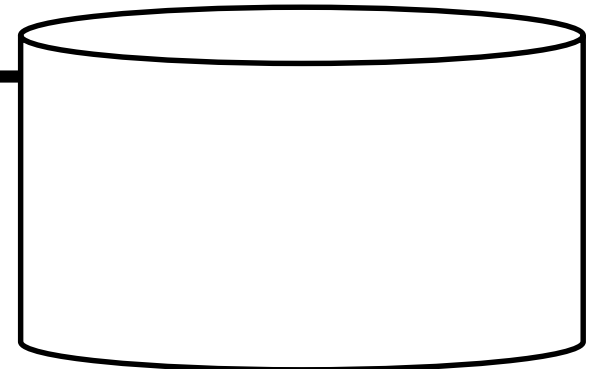
```
→ int main() {  
    int x;  
    return 0;  
}
```



Memória



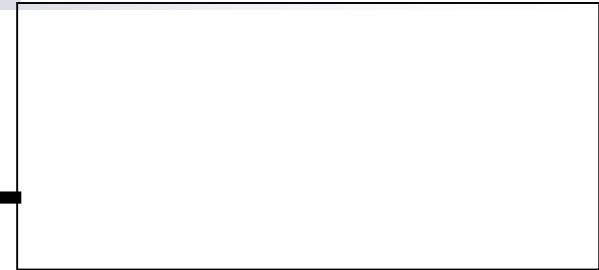
HD



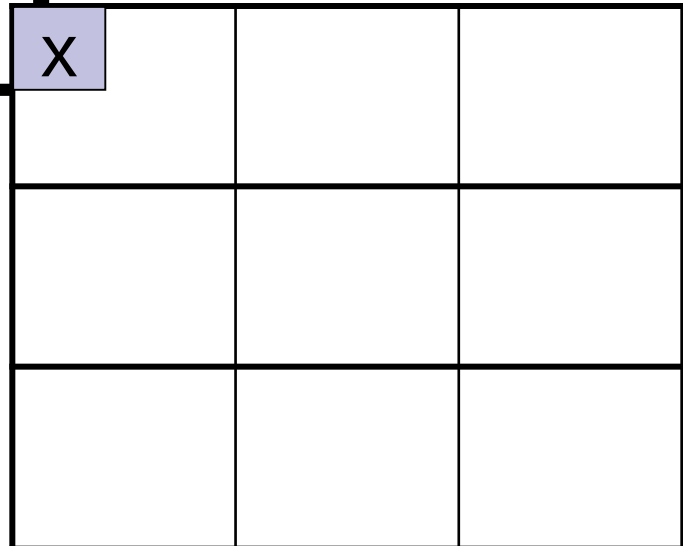
Execução

```
#include<stdio.h>
```

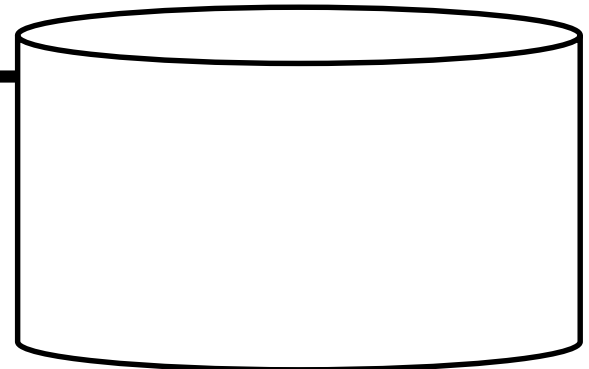
```
int main() {  
    → int x;  
    return 0;  
}
```



Memória



HD

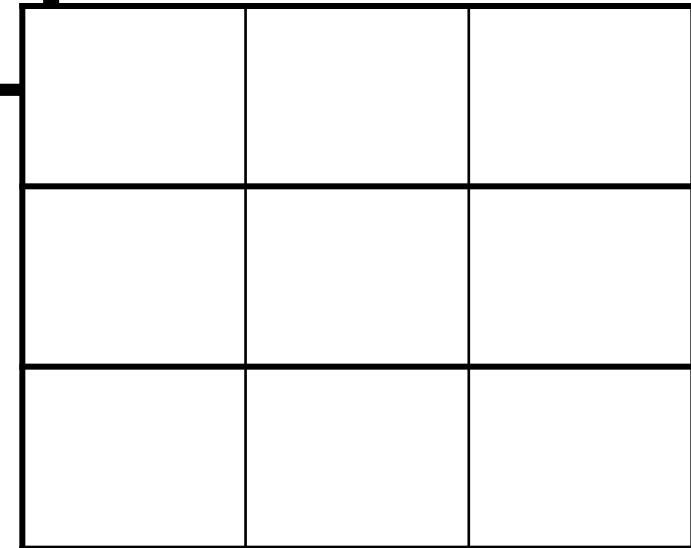


Execução

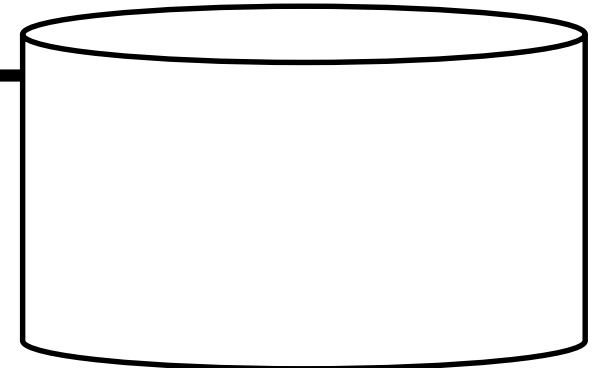
```
#include<stdio.h>
```

```
int main() {  
    int x;  
    → return 0;  
}
```

Memória



HD



Variáveis

- O programa anterior reserva uma área de memória chamada x, mas não a utiliza. Nenhum inteiro foi gravado em x.
- Existem 2 modos para gravar um número em uma área de memória
 - Atribuição
 - Função scanf

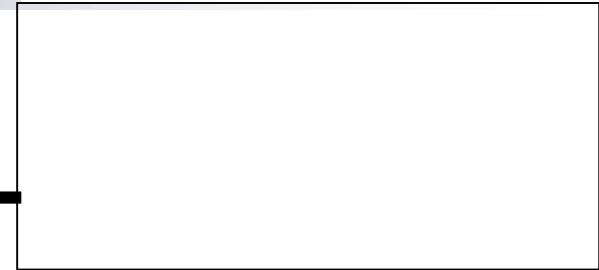


Atribuição

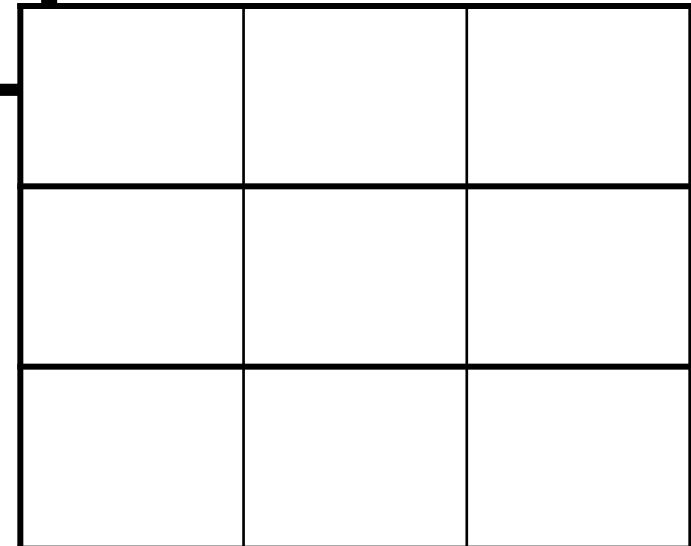
Execução

```
#include<stdio.h>
```

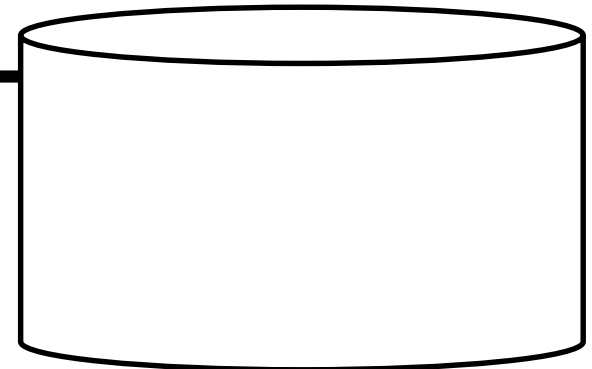
```
→ int main() {  
    int x;  
    x = 5;  
    return 0;  
}
```



Memória



HD



Execução

```
#include<stdio.h>
```

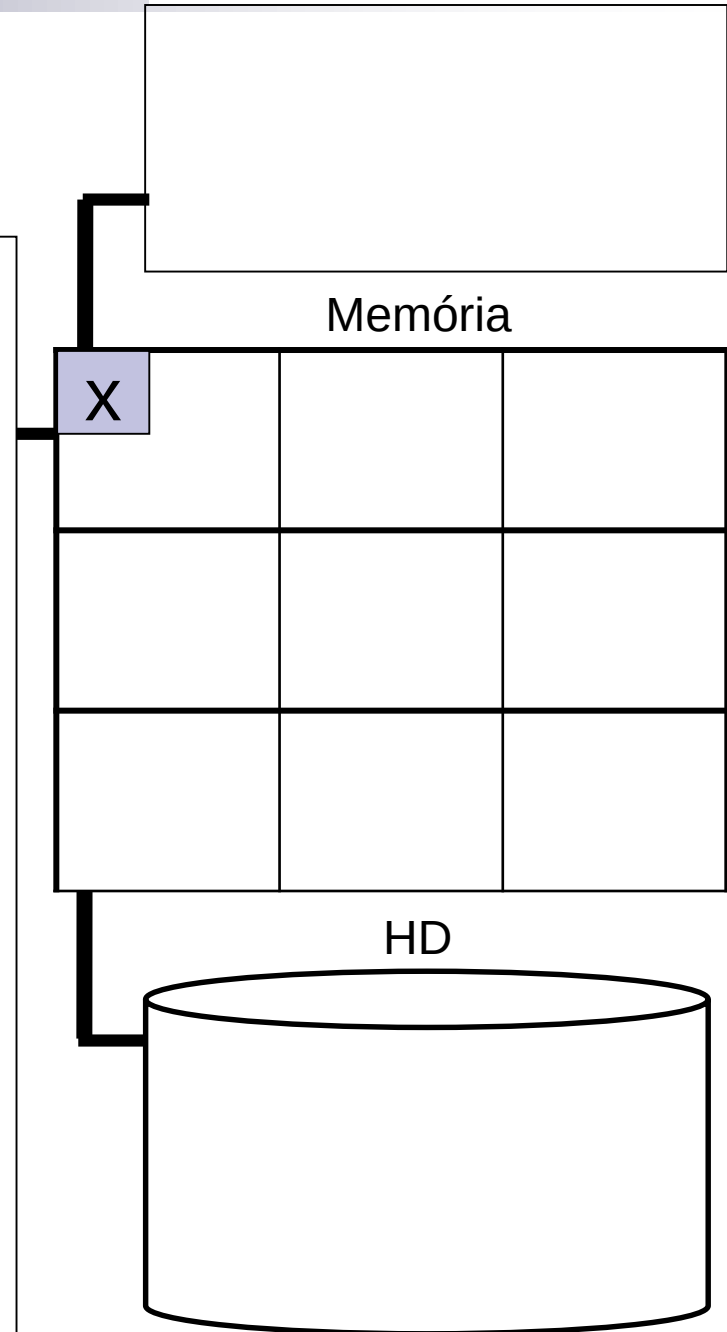
```
int main() {
```

```
→ int x;
```

```
    x = 5;
```

```
    return 0;
```

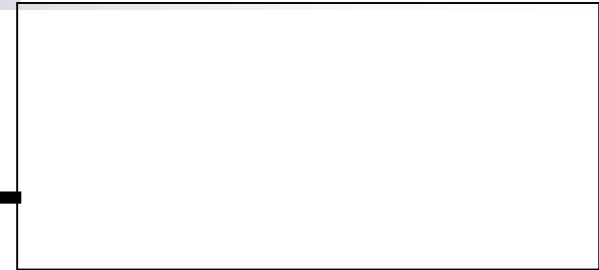
```
}
```



Execução

```
#include<stdio.h>
```

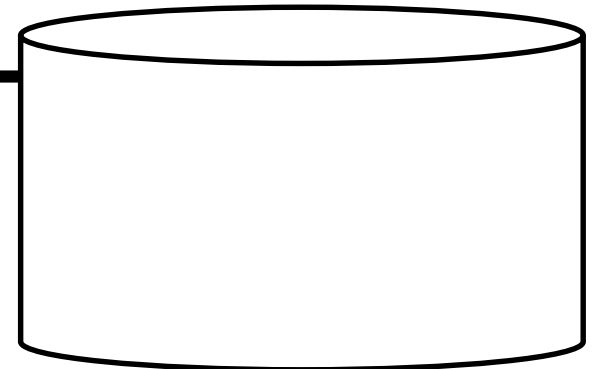
```
int main() {  
    int x;  
    → x = 5;  
    return 0;  
}
```



Memória

X	5		

HD

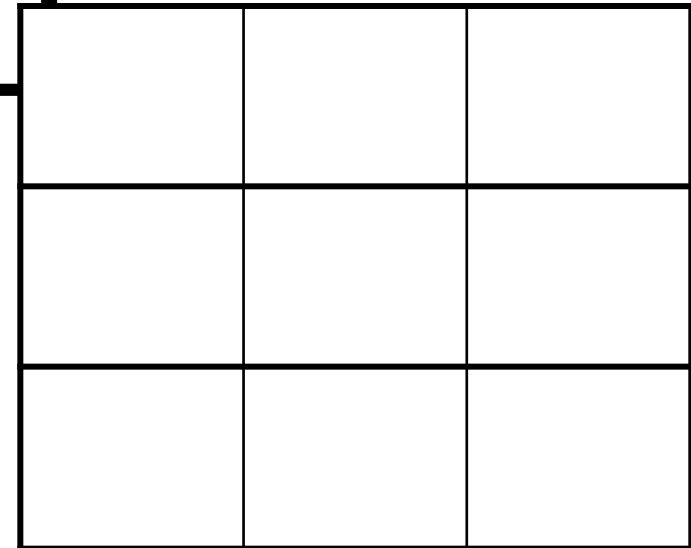


Execução

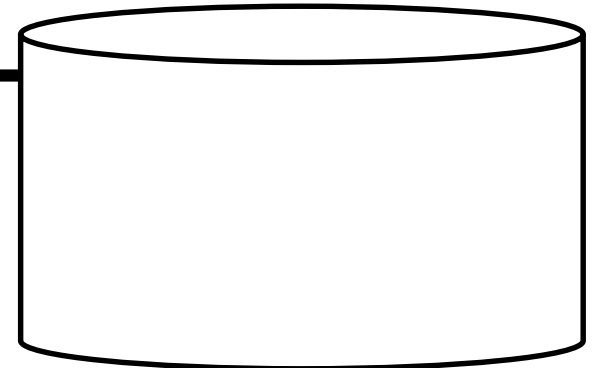
```
#include<stdio.h>
```

```
int main() {  
    int x;  
    x = 5;  
    → return 0;  
}
```

Memória



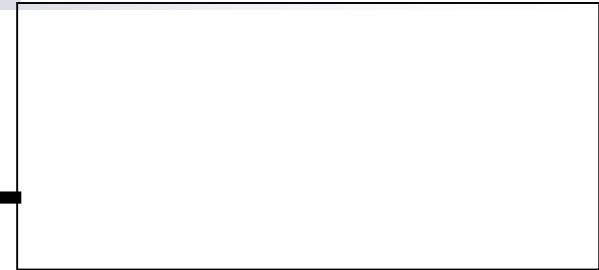
HD



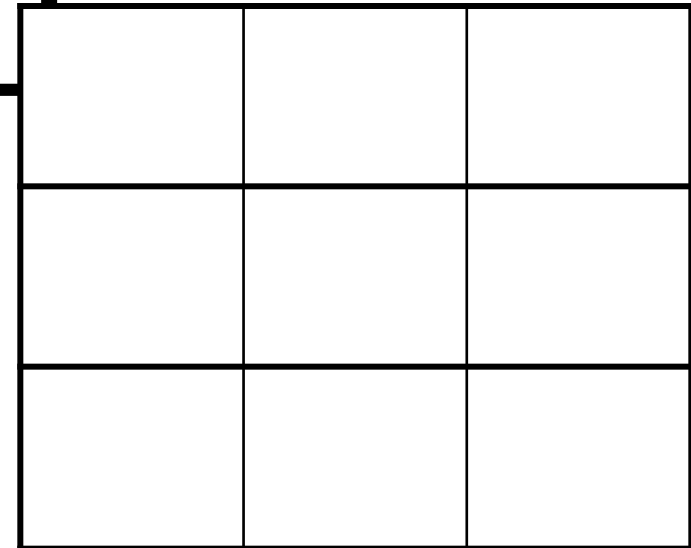
Execução

```
#include<stdio.h>
```

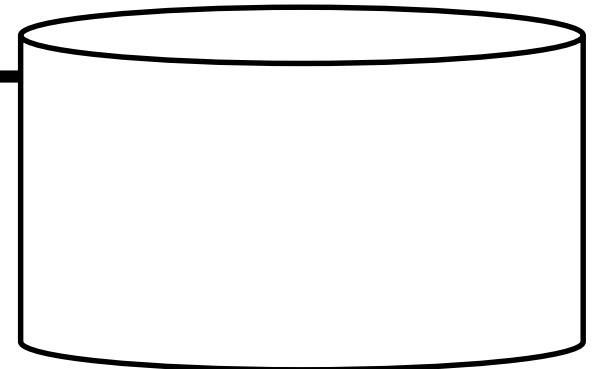
```
→ int main() {  
    int x;  
    x = 5;  
    x = 8;  
    x = 10;  
    return 0;  
}
```



Memória

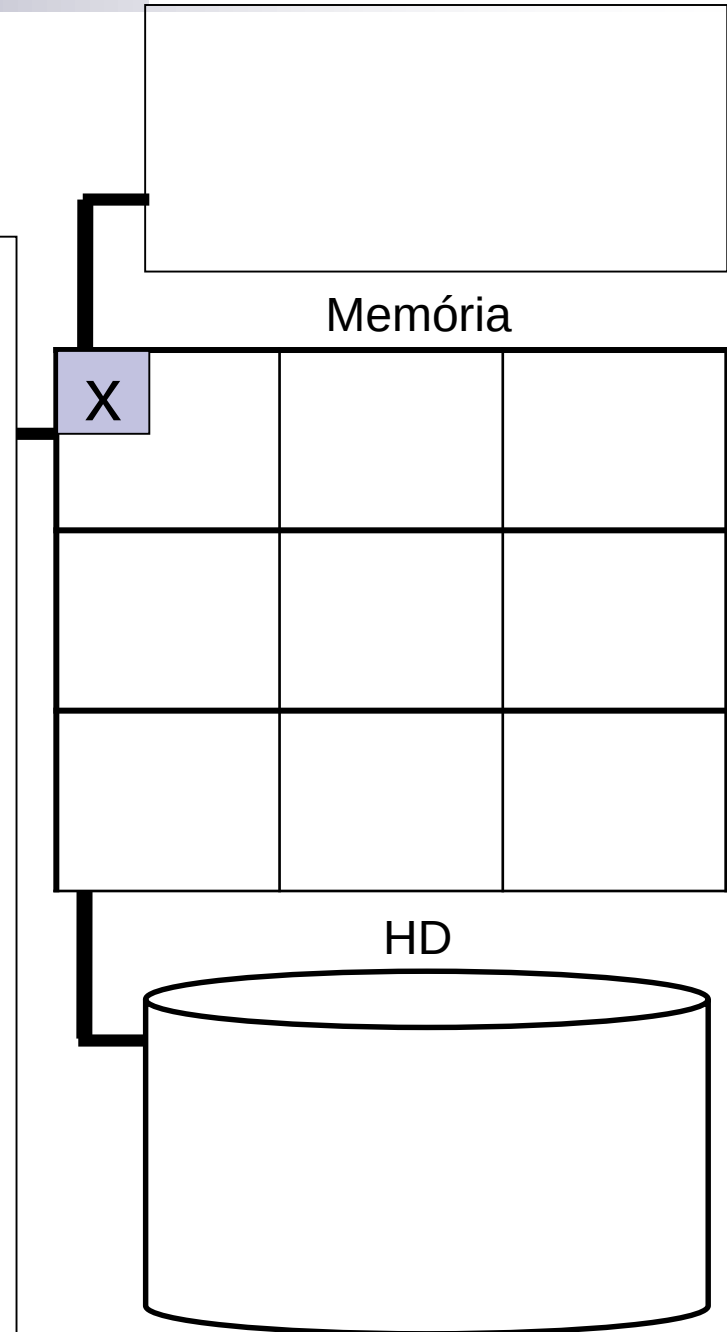


HD



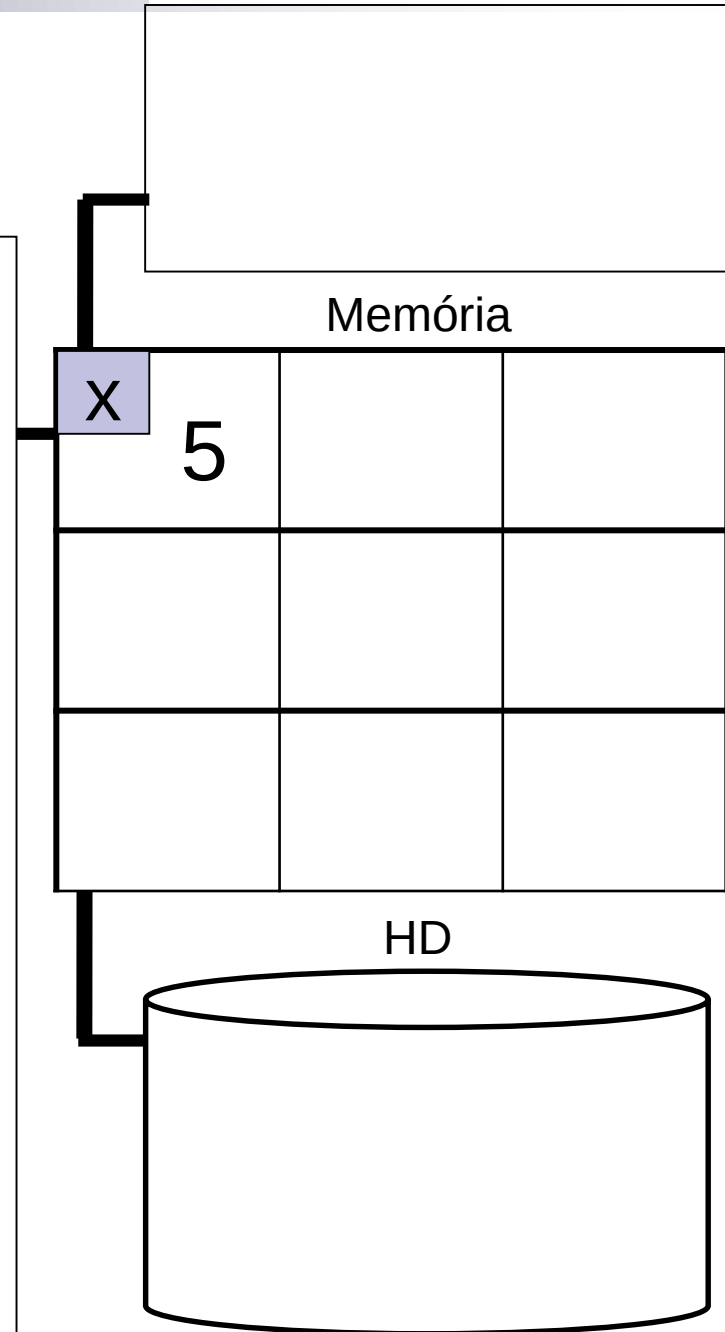
Execução

```
#include<stdio.h>
int main() {
    → int x;
      x = 5;
      x = 8;
      x = 10;
      return 0;
}
```



Execução

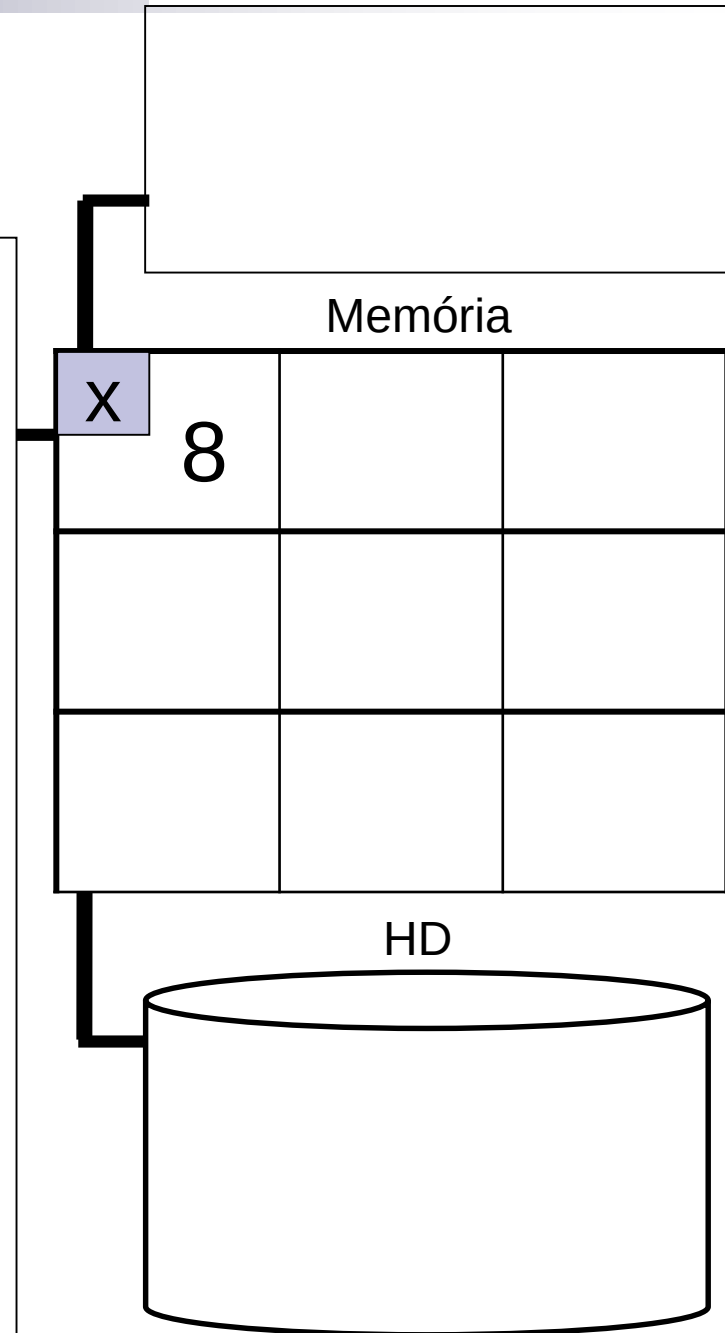
```
#include<stdio.h>
int main() {
    int x;
    → x = 5;
    x = 8;
    x = 10;
    return 0;
}
```



Execução

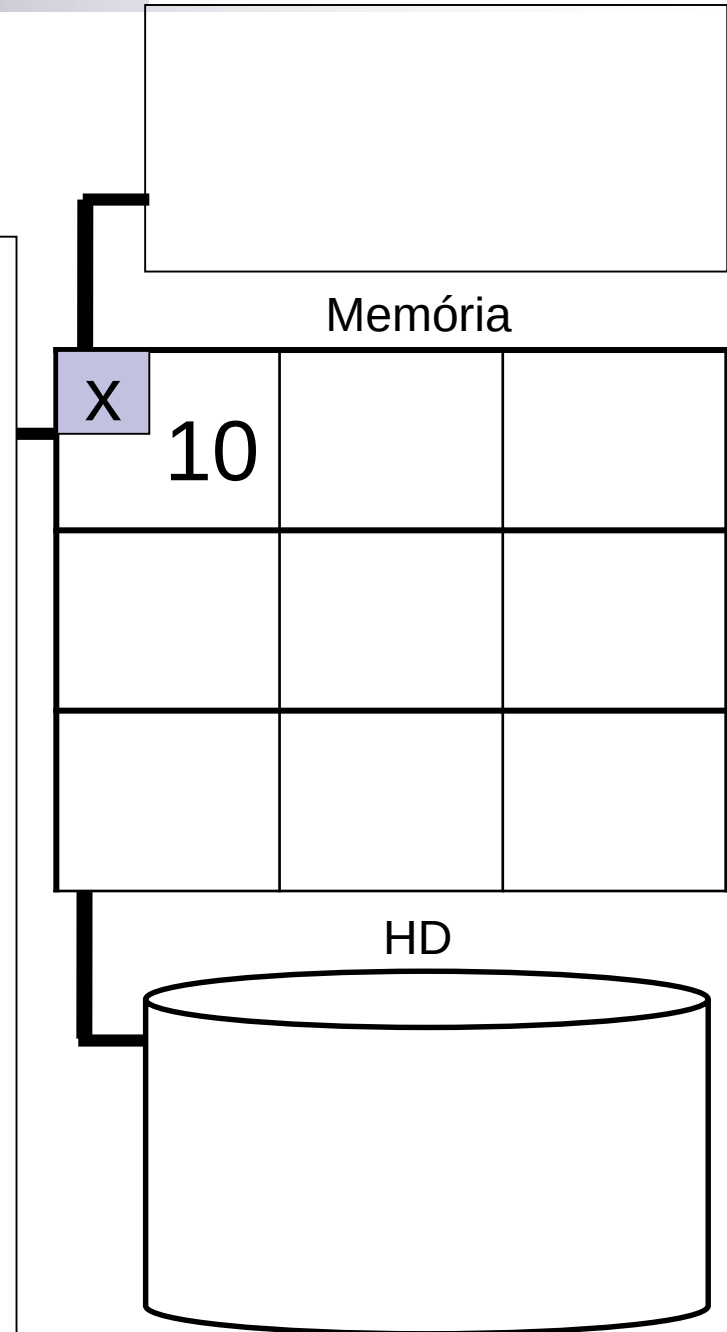
```
#include<stdio.h>
int main() {
    int x;
    x = 5;
    → x = 8;
    x = 10;
    return 0;
}
```

Note que a atribuição $x = 8$ apagou o 5 que estava em x e gravou o 8 por cima.



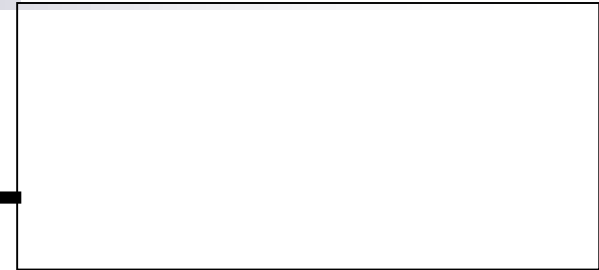
Execução

```
#include<stdio.h>
int main() {
    int x;
    x = 5;
    x = 8;
    → x = 10;
    return 0;
}
```

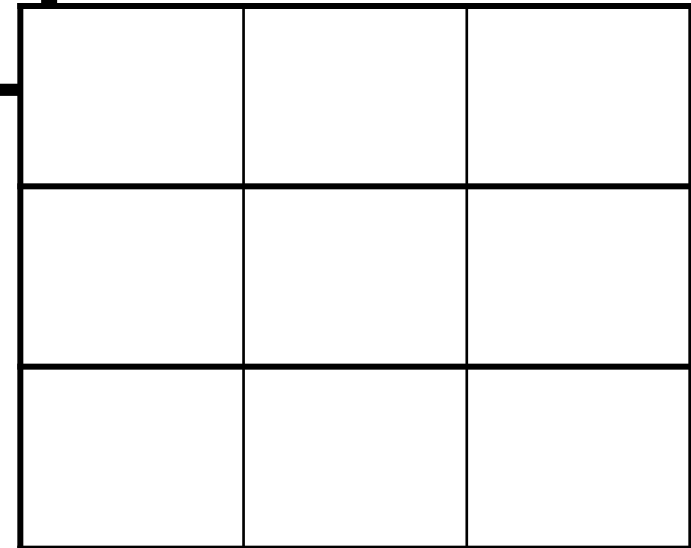


Execução

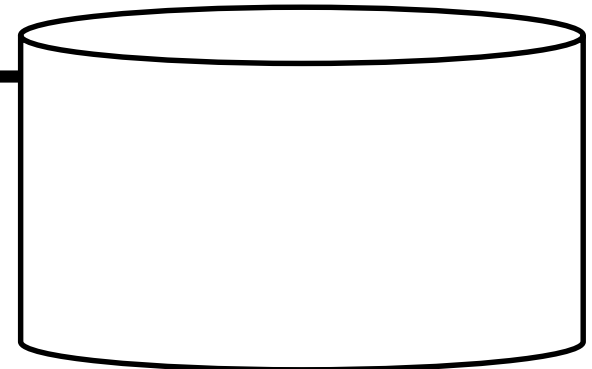
```
#include<stdio.h>
int main() {
    int x;
    x = 5;
    x = 8;
    x = 10;
    → return 0;
}
```



Memória



HD

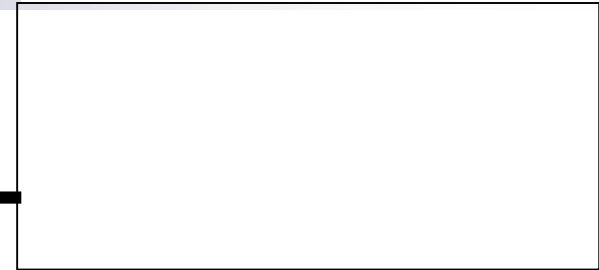


Execução

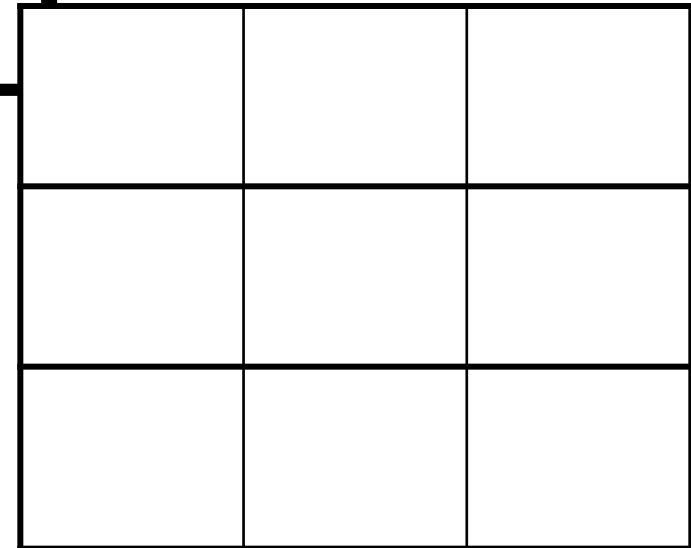
```
#include<stdio.h>
```



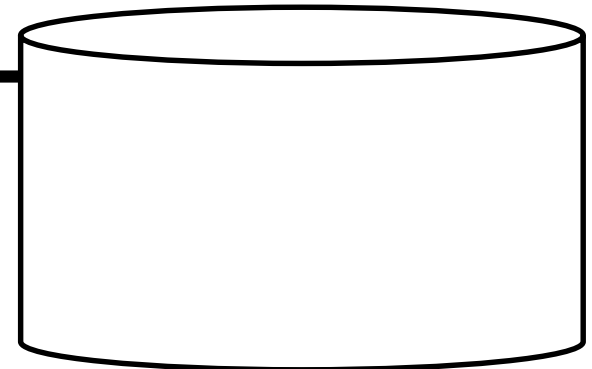
```
int main() {  
    int x; int y; int z, w;  
    return 0;  
}
```



Memória



HD

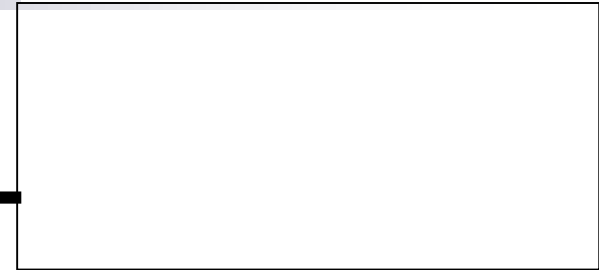


Execução

```
#include<stdio.h>
```

```
int main() {
```

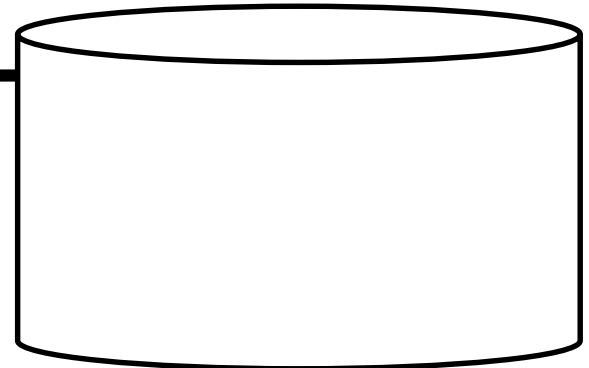
```
→ int x; int y; int z, w;  
   return 0;  
}
```



Memória

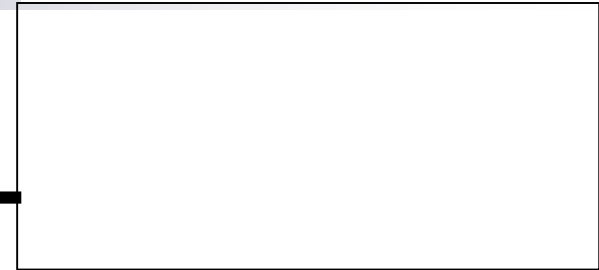
x	y	z
w		

HD

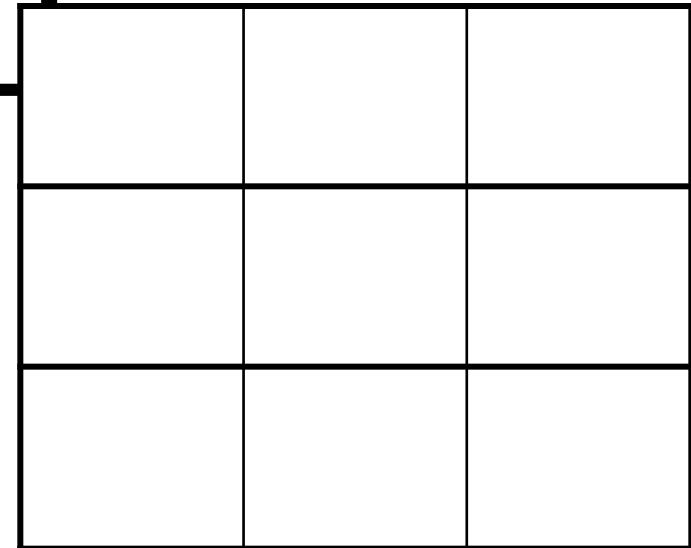


Execução

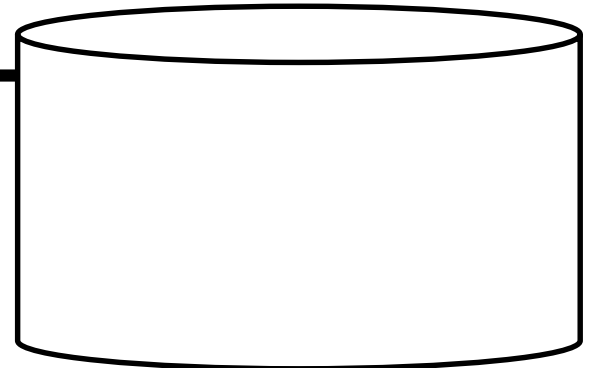
```
#include<stdio.h>
int main() {
    int x; int y; int z, w;
    → return 0;
}
```



Memória



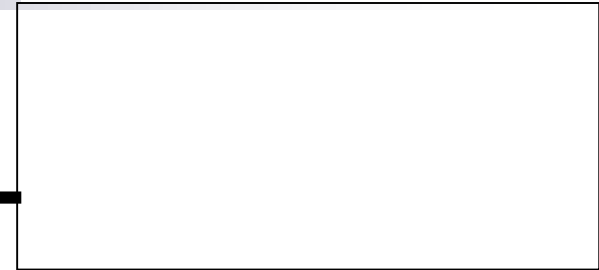
HD



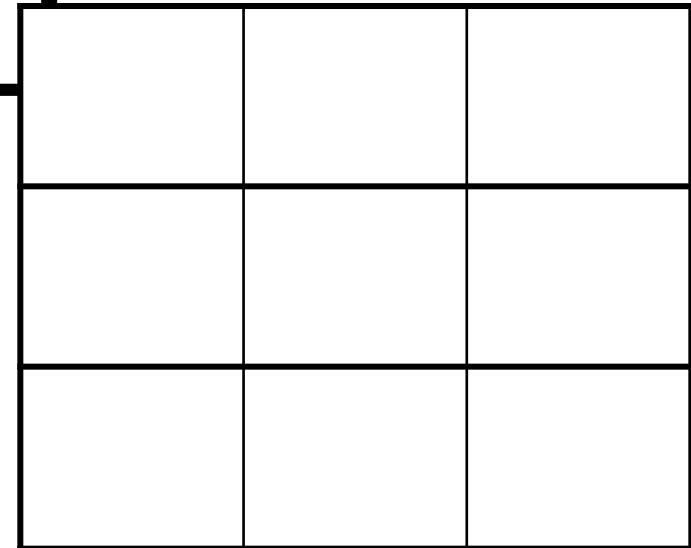
Execução

```
#include<stdio.h>
```

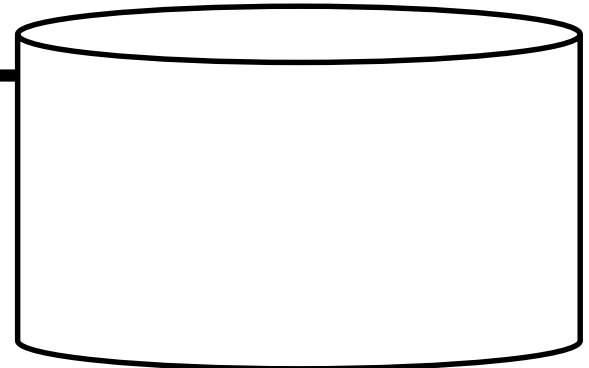
```
→ int main() {  
    int x; int y;  
    x = 5;  
    y = 8;  
    return 0;  
}
```



Memória

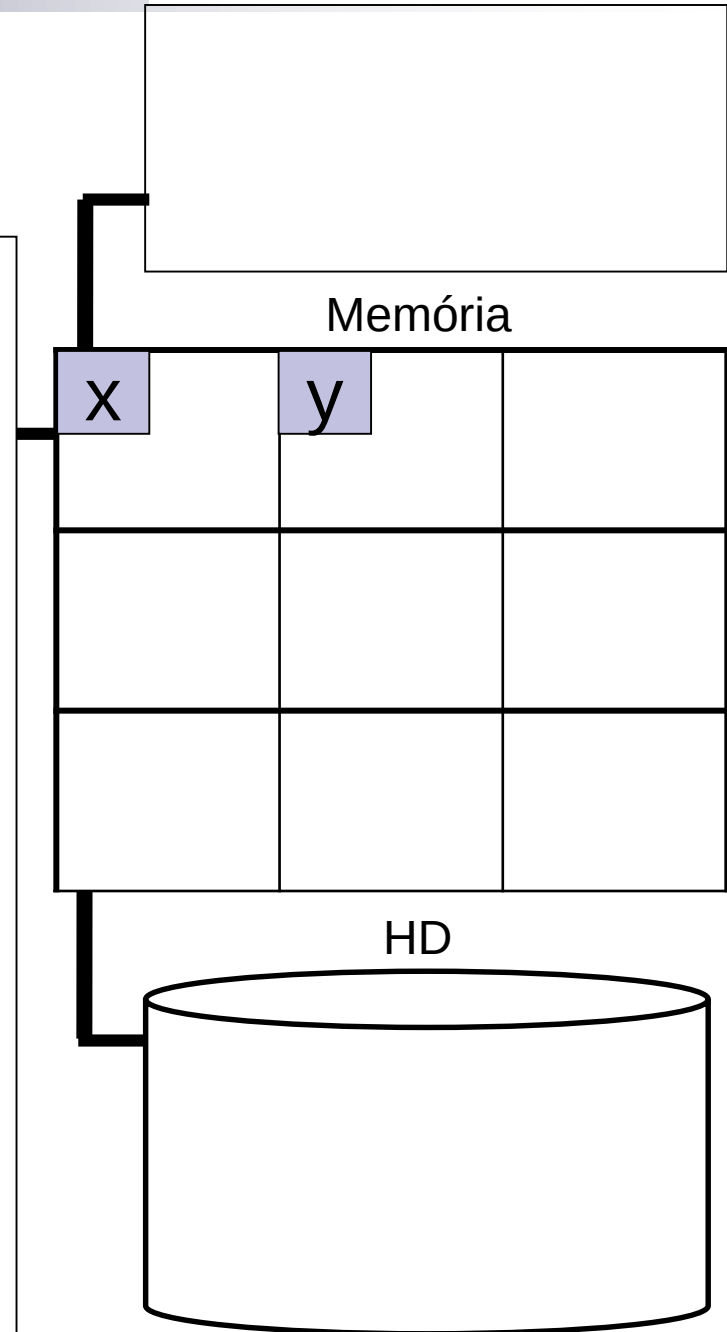


HD



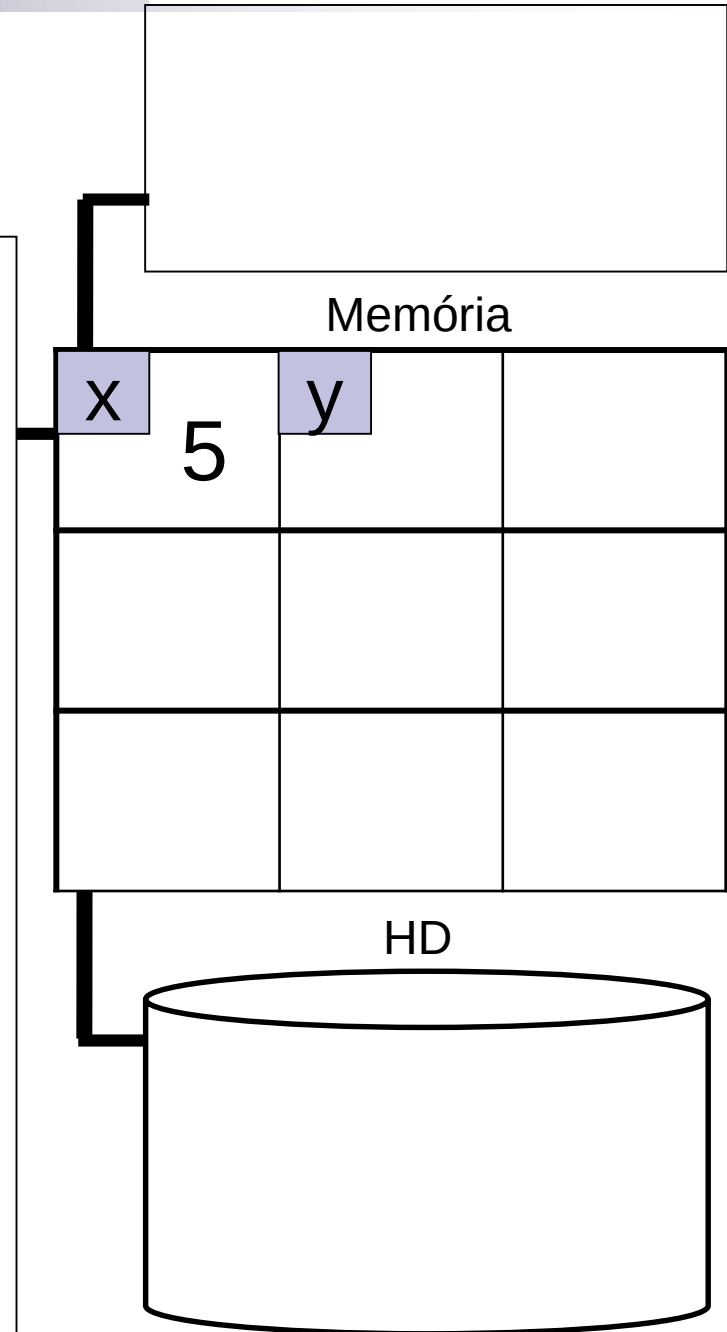
Execução

```
#include<stdio.h>
int main() {
    → int x; int y;
      x = 5;
      y = 8;
      return 0;
}
```



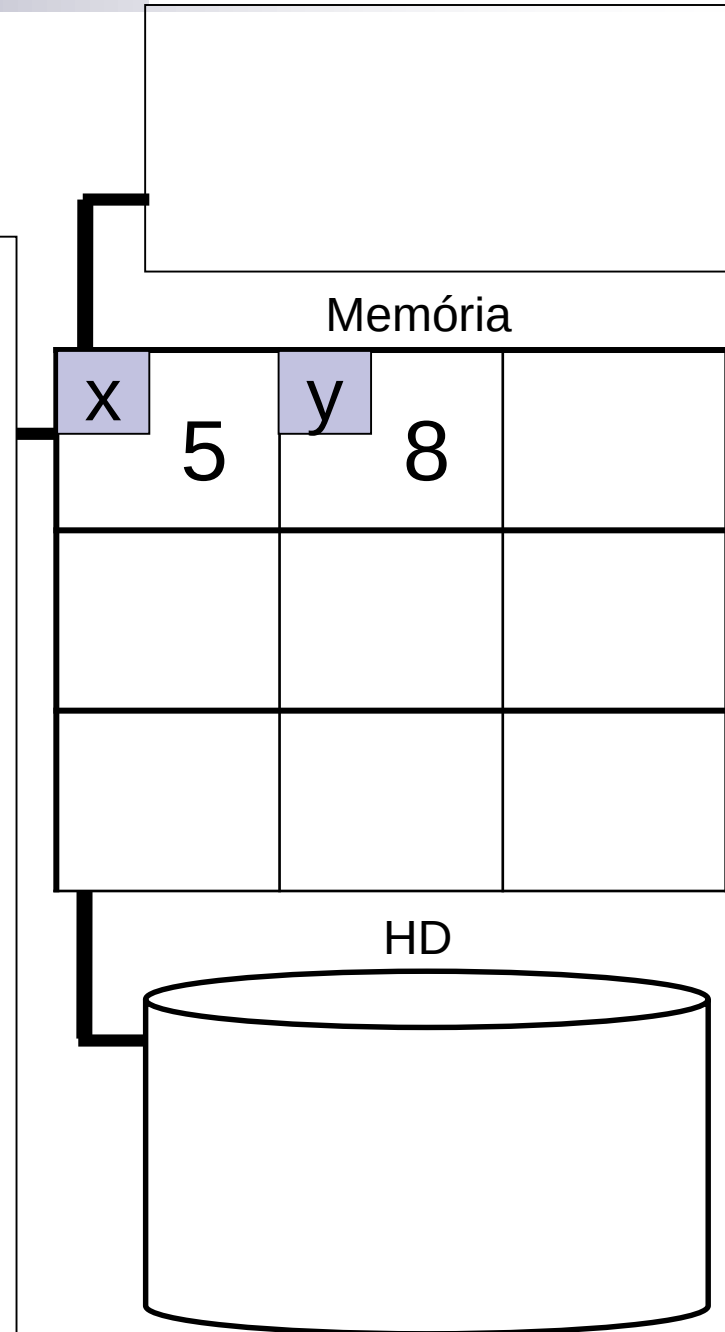
Execução

```
#include<stdio.h>
int main() {
    int x; int y;
    → x = 5;
      y = 8;
    return 0;
}
```



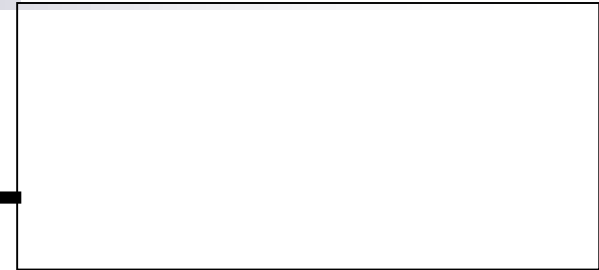
Execução

```
#include<stdio.h>
int main() {
    int x; int y;
    x = 5;
    → y = 8;
    return 0;
}
```

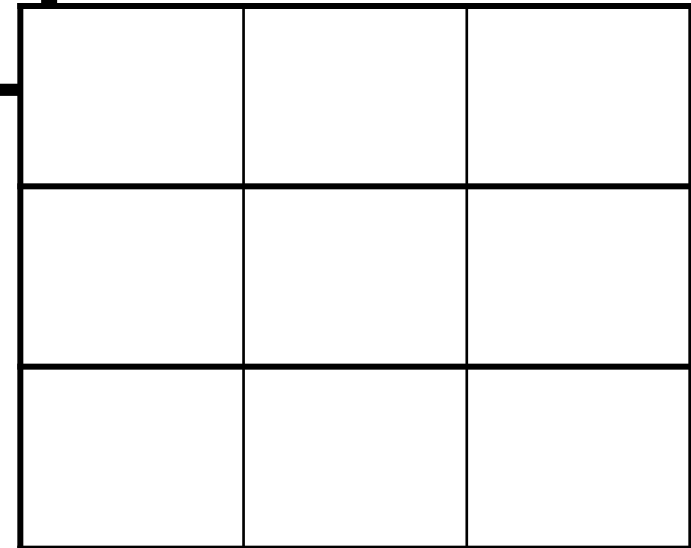


Execução

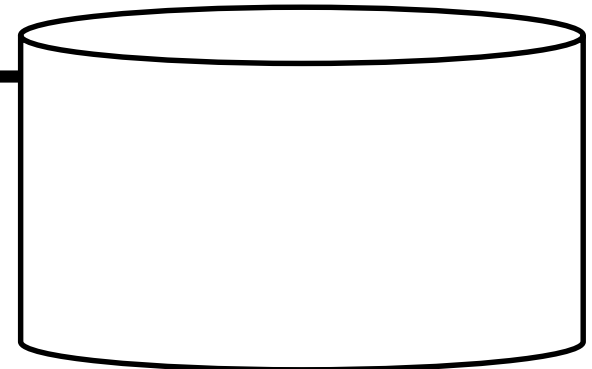
```
#include<stdio.h>
int main() {
    int x; int y;
    x = 5;
    y = 8;
    → return 0;
}
```



Memória

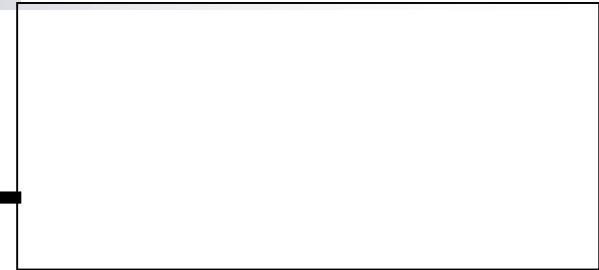


HD

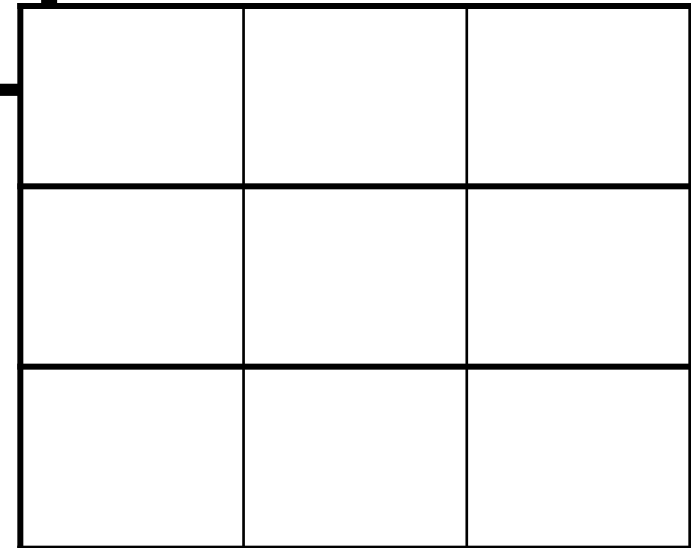


Execução

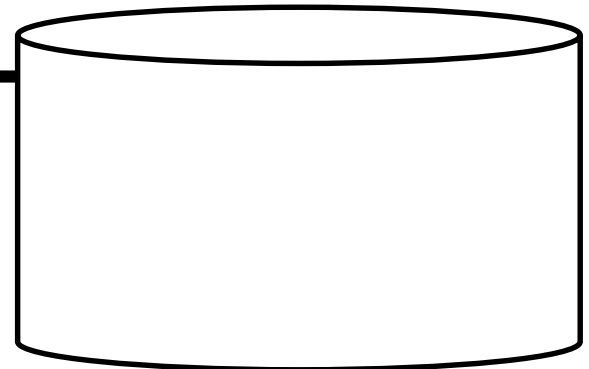
```
#include<stdio.h>
→ int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```



Memória

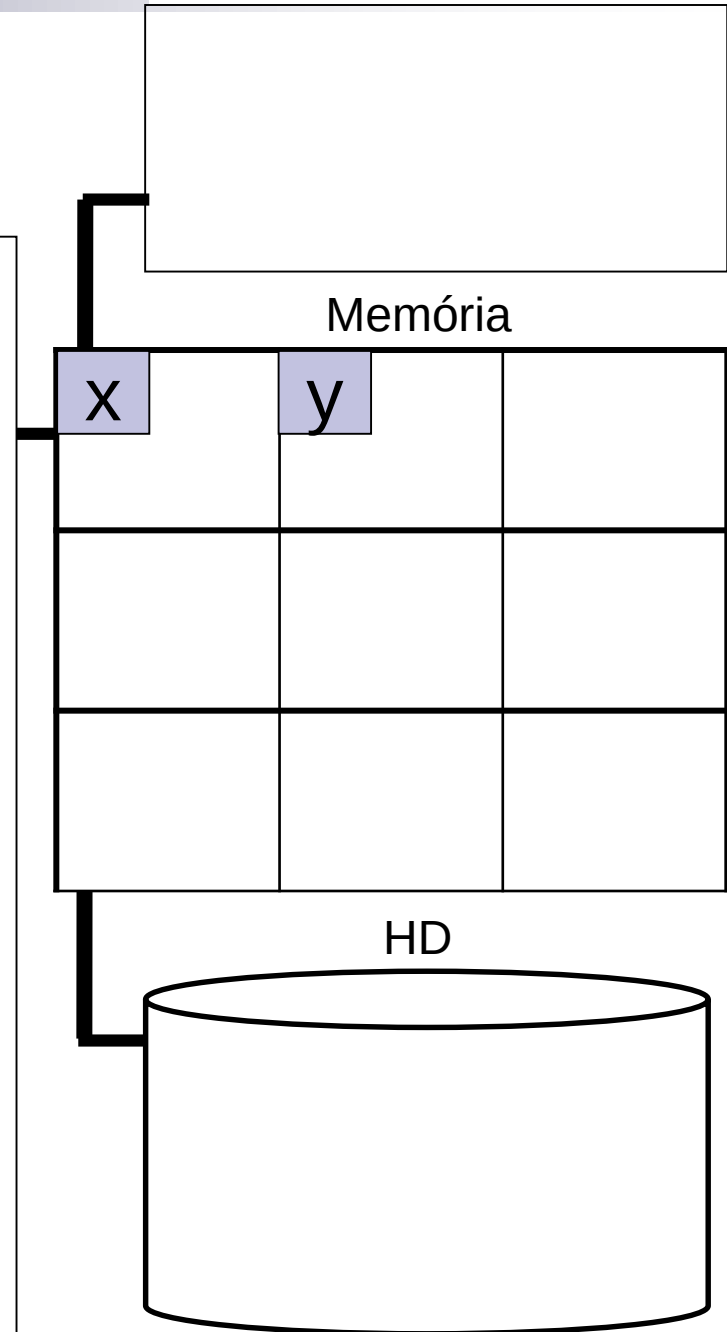


HD



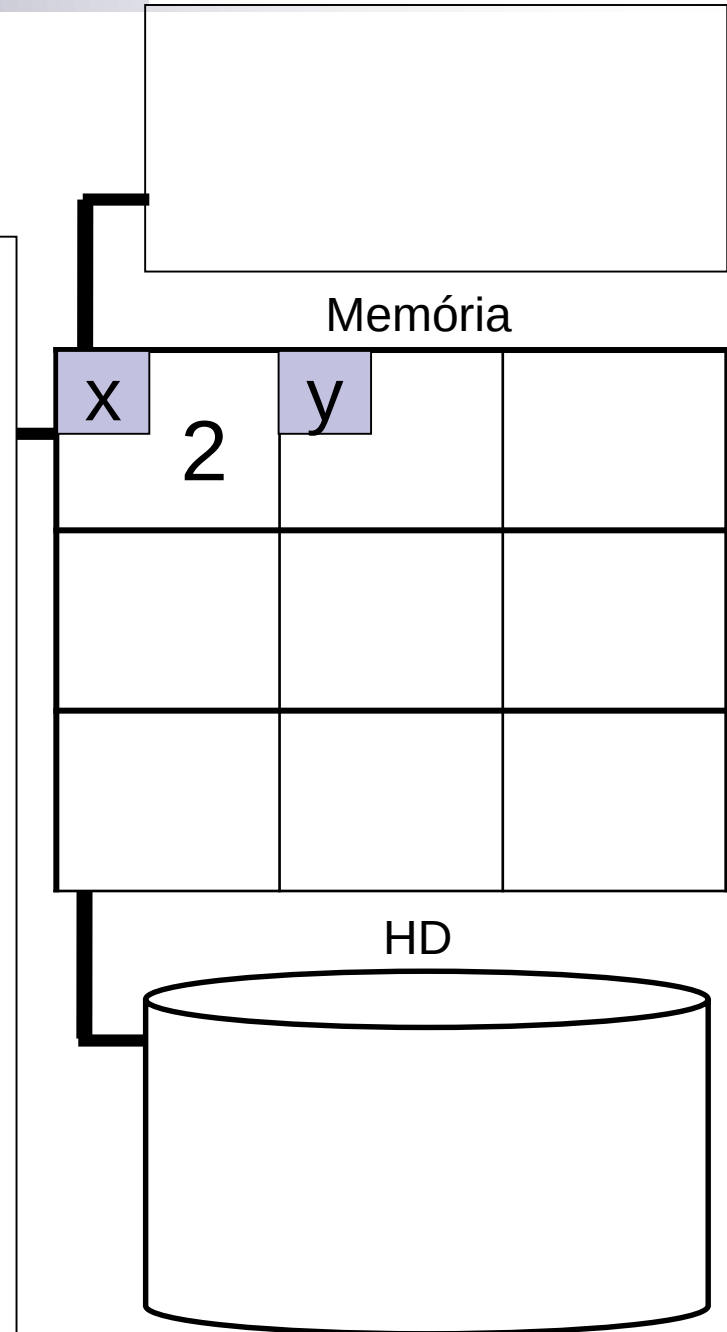
Execução

```
#include<stdio.h>
int main() {
    → int x; int y;
      x = 2;
      y = x + 3;
      x = y - x;
      return 0;
}
```



Execução

```
#include<stdio.h>
int main() {
    int x; int y;
    → x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```

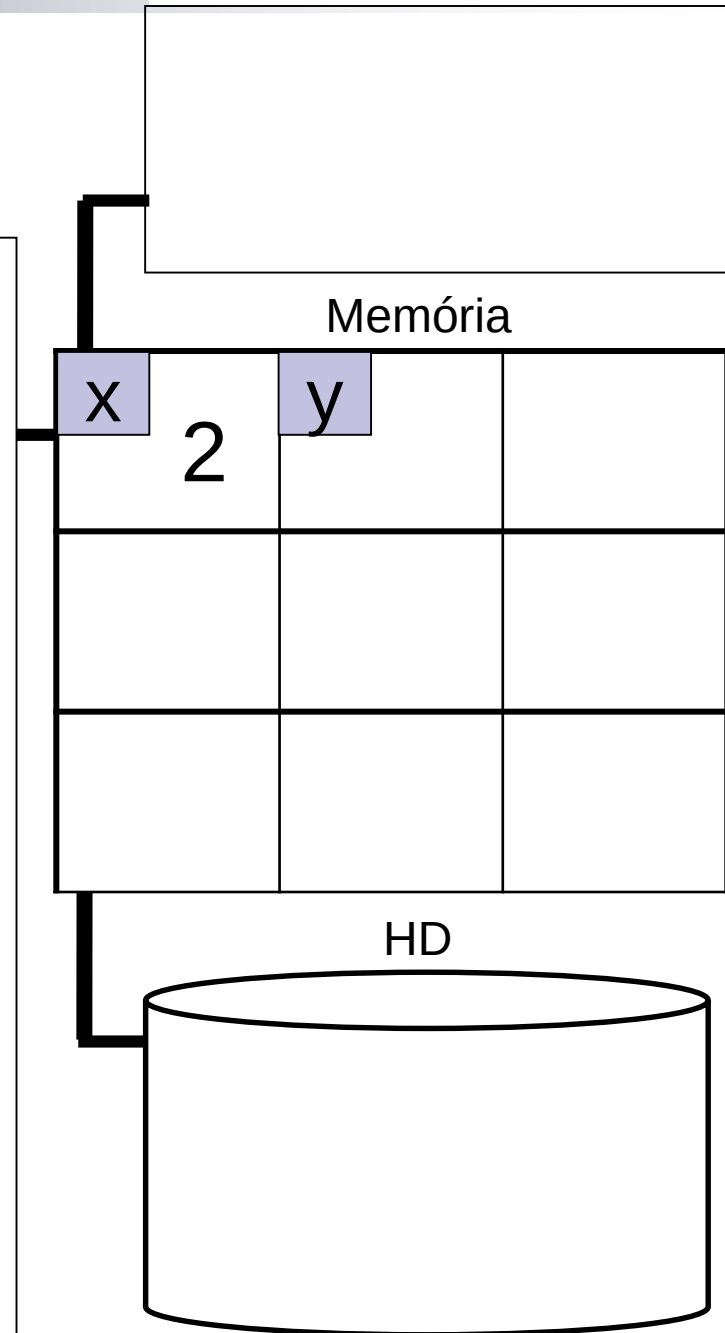


Execução

```
#include<stdio.h>
int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```

*Primeiro, o computador calcula,
em uma memória de rascunho,
 $x + 3$. Ou seja, ele calcula*

*$x + 3$
 $= 2 + 3$
 $= 5$*



Execução

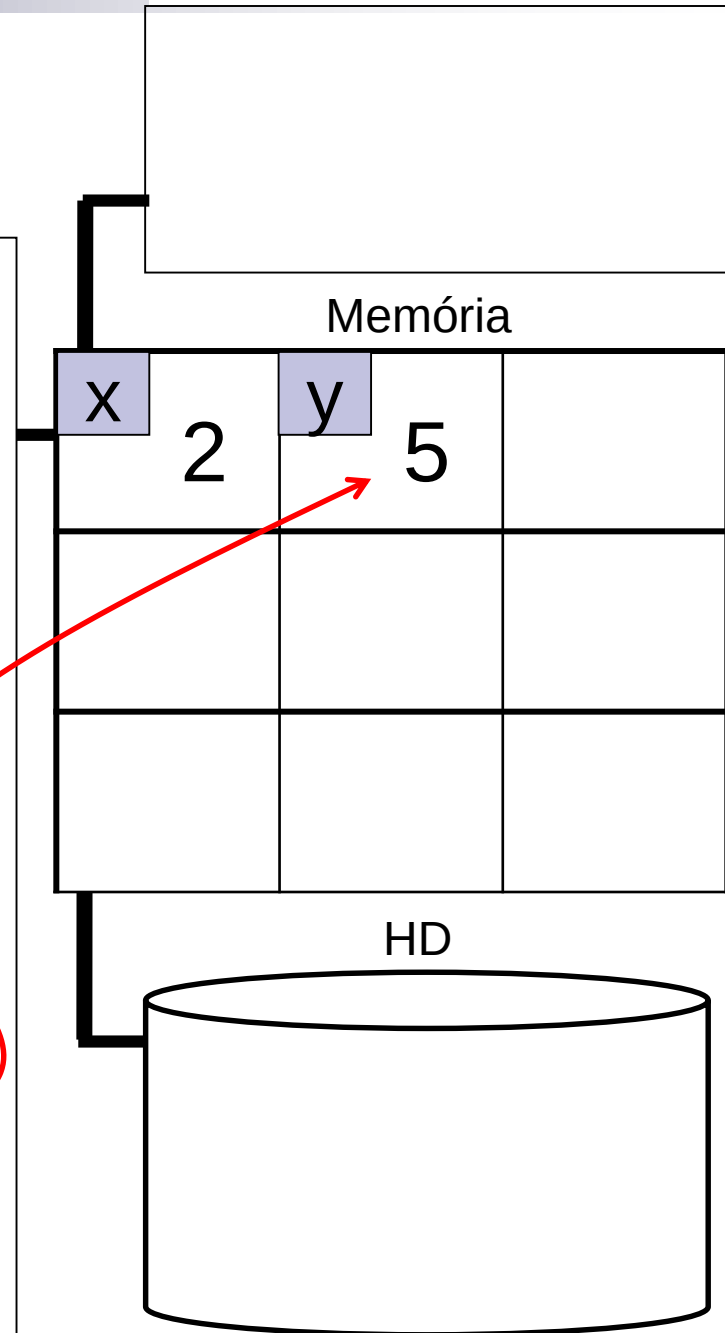
```
#include<stdio.h>
int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```

Primeiro, o computador calcula, em uma memória de rascunho, $x + 3$. Ou seja, ele calcula

$x + 3$

$= 2 + 3$

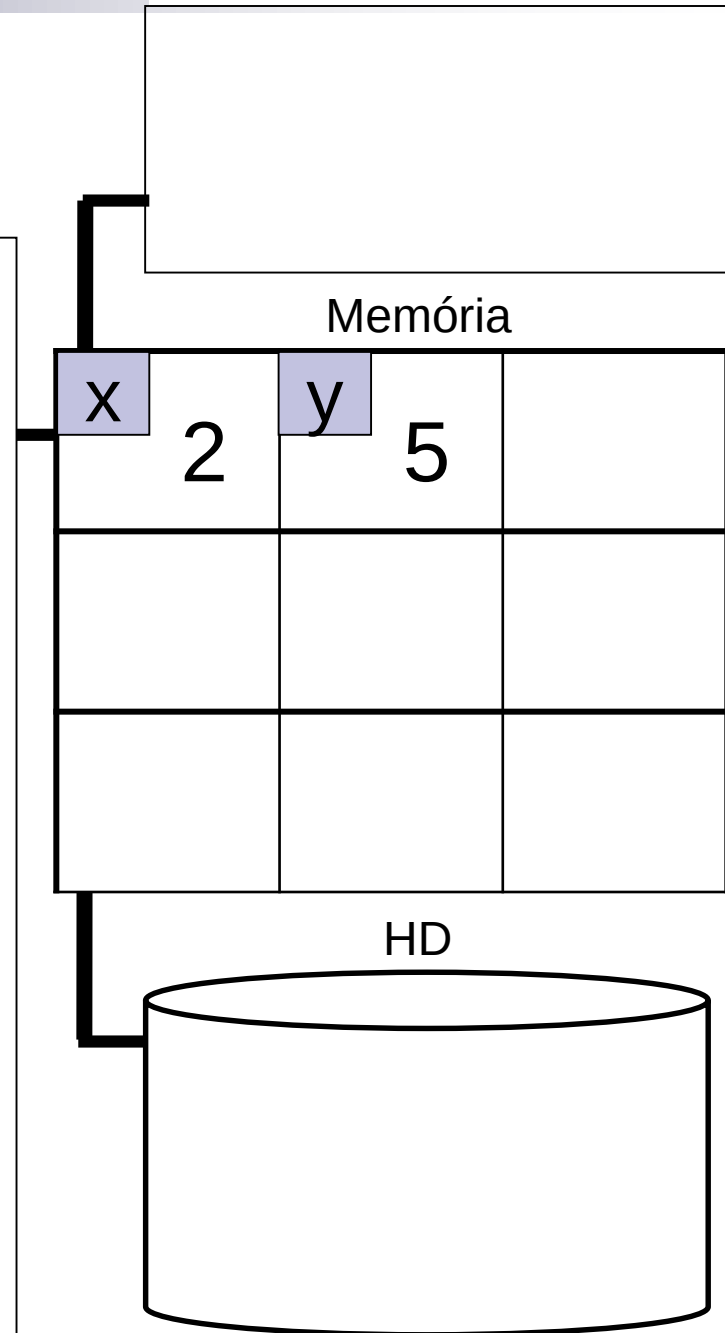
$= 5$ Depois, o 5 vai para a memória.



Execução

```
#include<stdio.h>
int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```

*Primeiro, o computador calcula,
em uma memória de rascunho,
 $y - x$. Ou seja, ele calcula*

$$\begin{aligned} y - x \\ = 5 - 2 \\ = 3 \end{aligned}$$


Execução

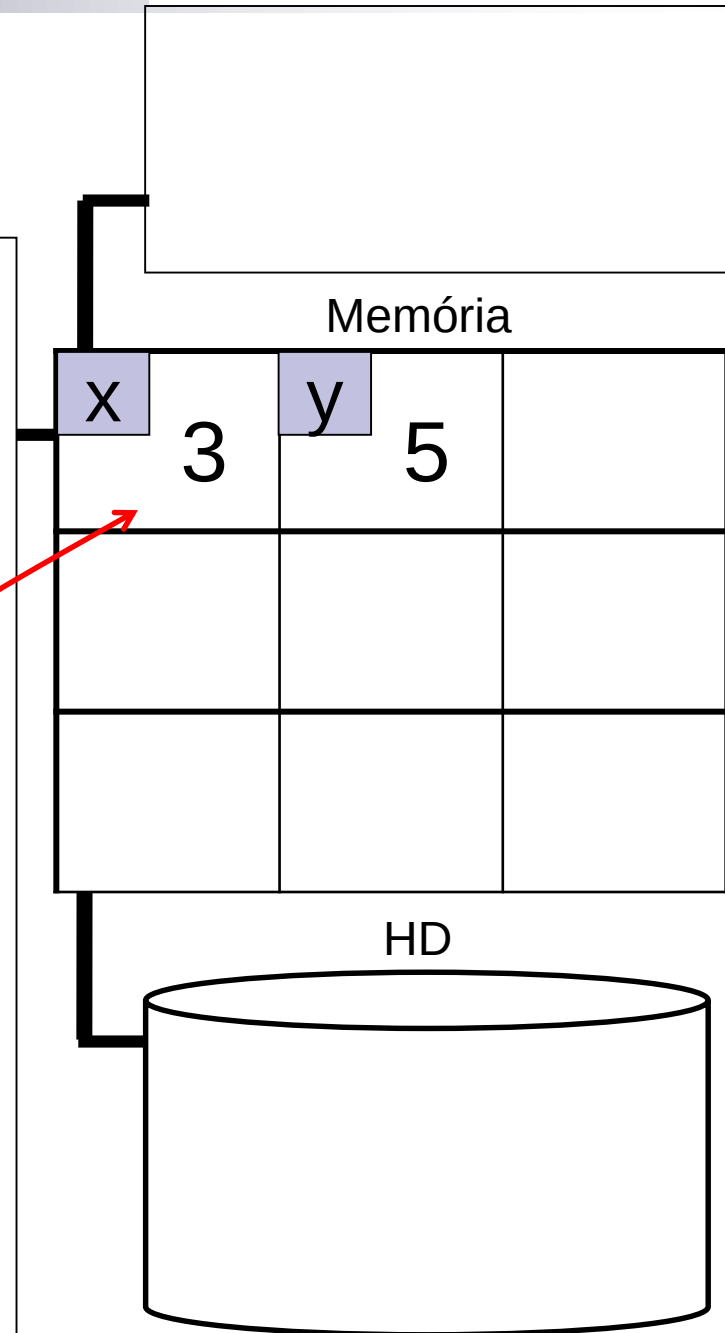
```
#include<stdio.h>
int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```

Primeiro, o computador calcula,
em uma memória de rascunho,
 $y - x$. Ou seja, ele calcula

$y - x$

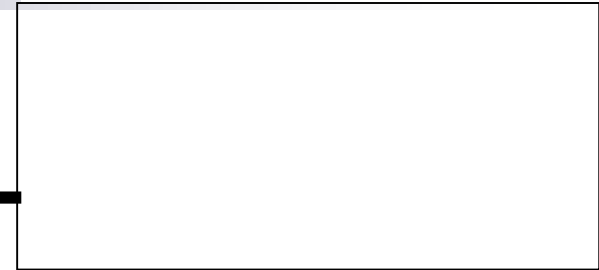
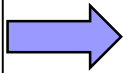
$= 5 - 2$

$= 3$ Depois, o 3 vai para a
memória.

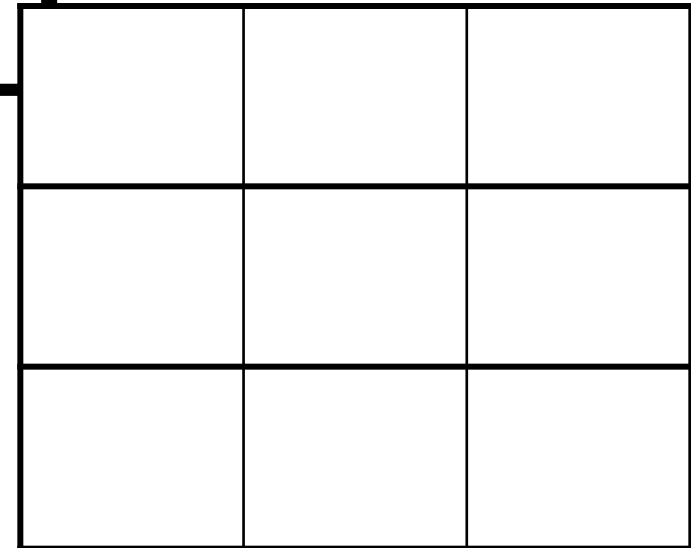


Execução

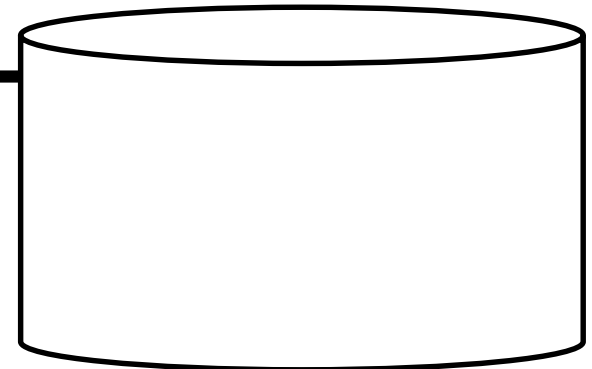
```
#include<stdio.h>
int main() {
    int x; int y;
    x = 2;
    y = x + 3;
    x = y - x;
    return 0;
}
```



Memória

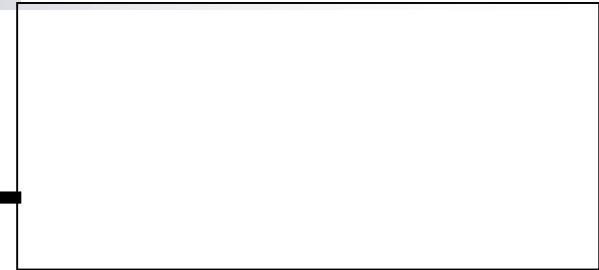


HD

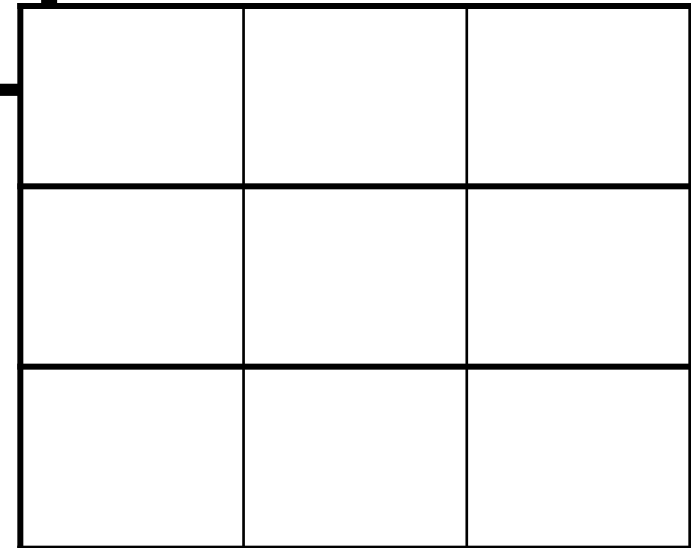


Execução

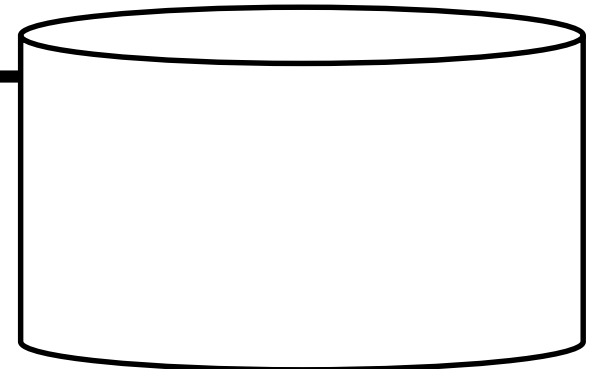
```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    y = 6;
    temp = x;
    x = y;
    y = temp;
    return 0;
}
```



Memória



HD



Execução

```
#include<stdio.h>
```

```
int main() {
```

```
→ int x; int y; int temp;
```

```
  x = 2;
```

```
  y = 6;
```

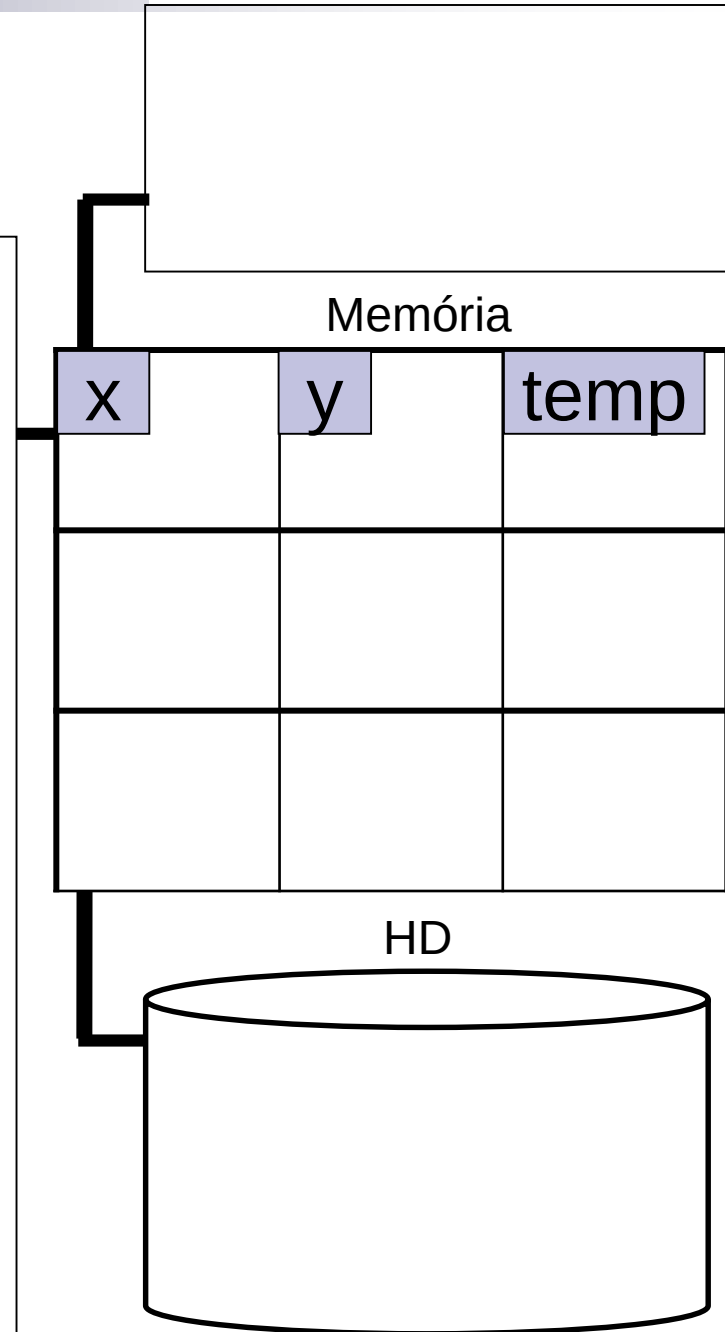
```
  temp = x;
```

```
  x = y;
```

```
  y = temp;
```

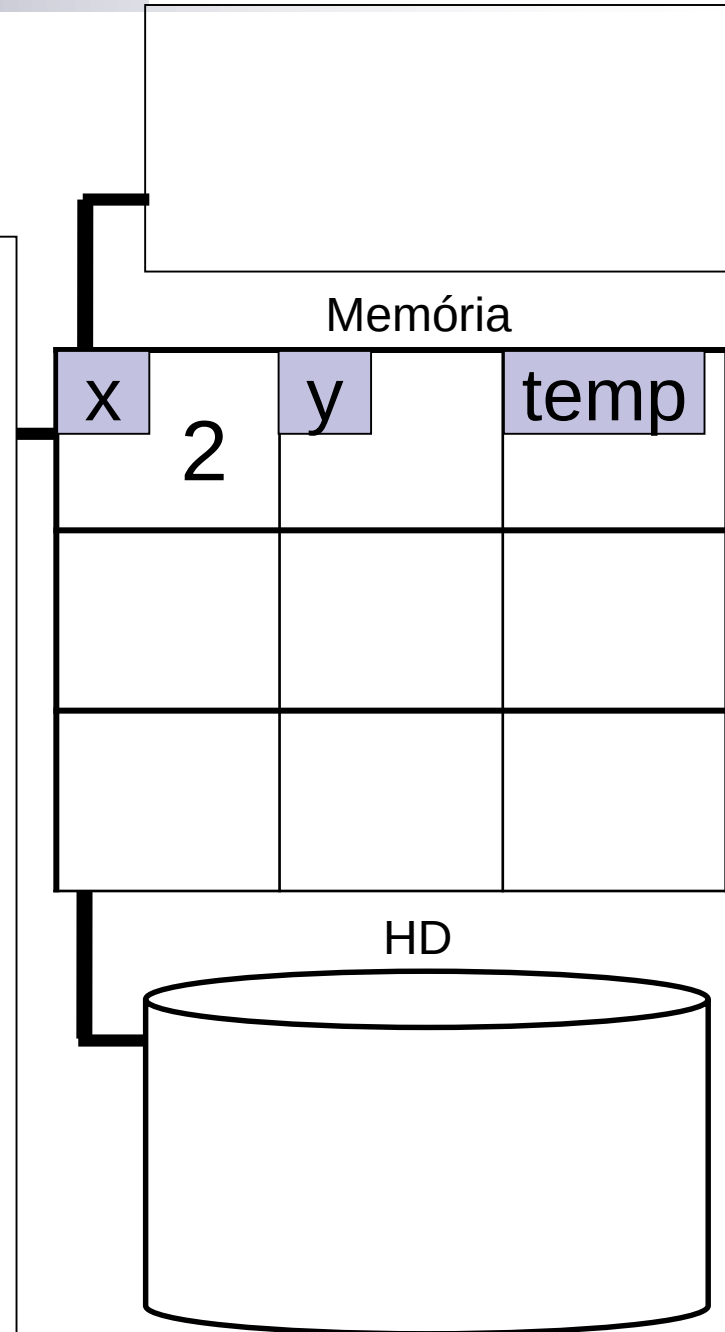
```
  return 0;
```

```
}
```



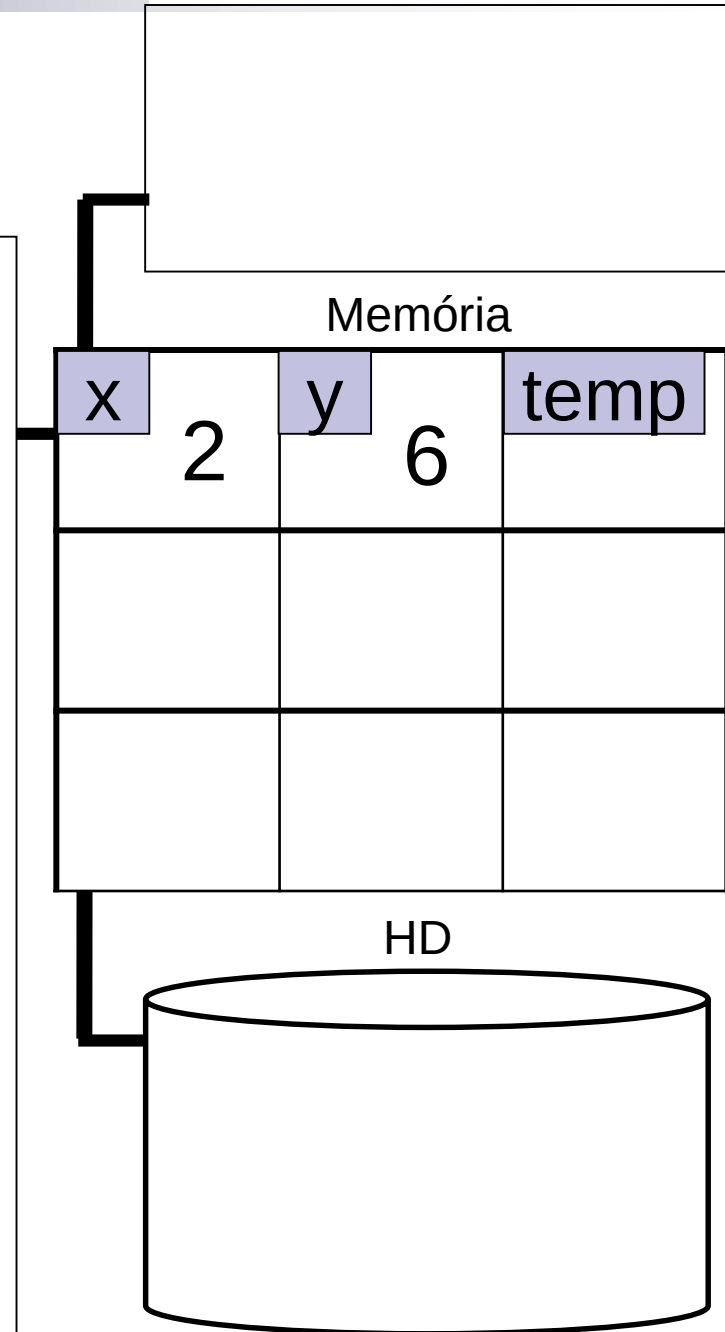
Execução

```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    → x = 2;
      y = 6;
      temp = x;
      x = y;
      y = temp;
      return 0;
}
```



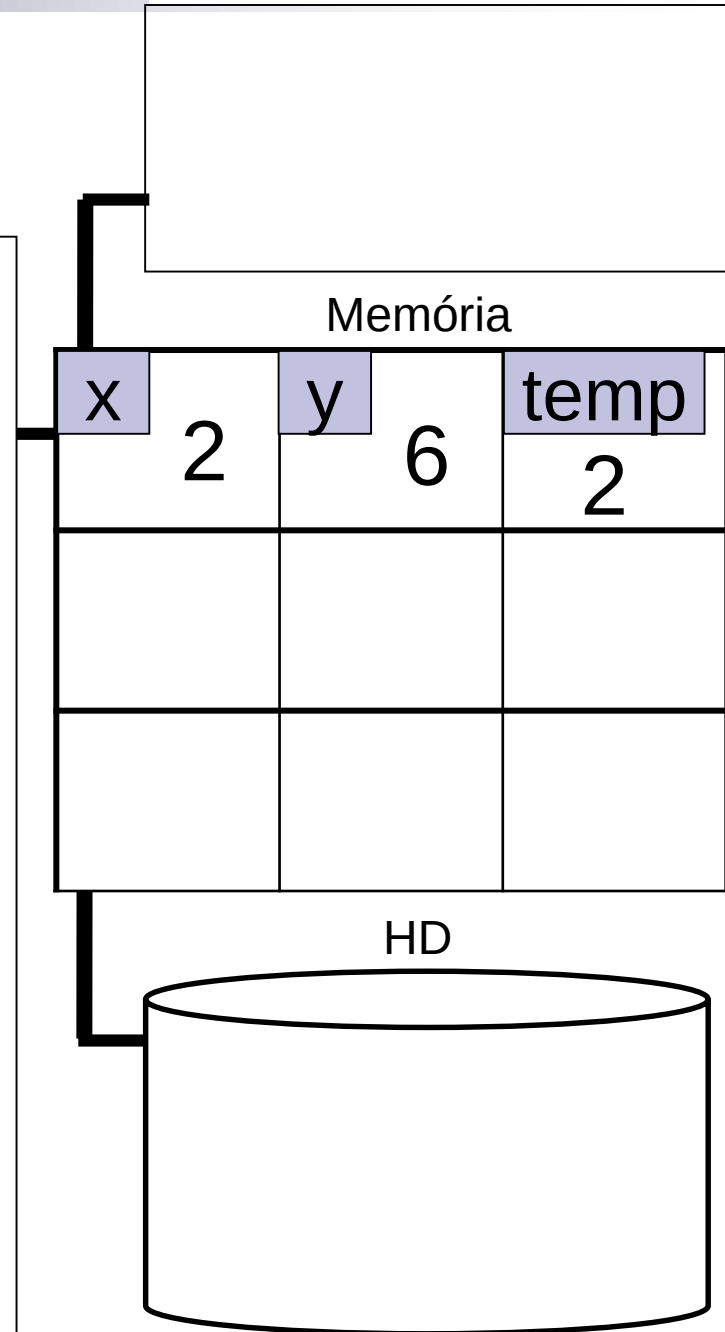
Execução

```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    → y = 6;
    temp = x;
    x = y;
    y = temp;
    return 0;
}
```



Execução

```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    y = 6;
    → temp = x;
    x = y;
    y = temp;
    return 0;
}
```



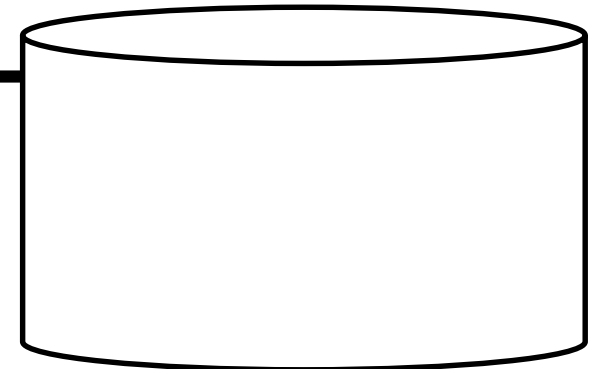
Execução

```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    y = 6;
    temp = x;
    x = y;
    y = temp;
    return 0;
}
```

Memória

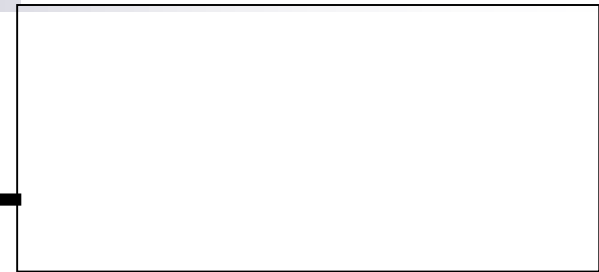
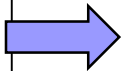
x	y	temp
6	6	2

HD



Execução

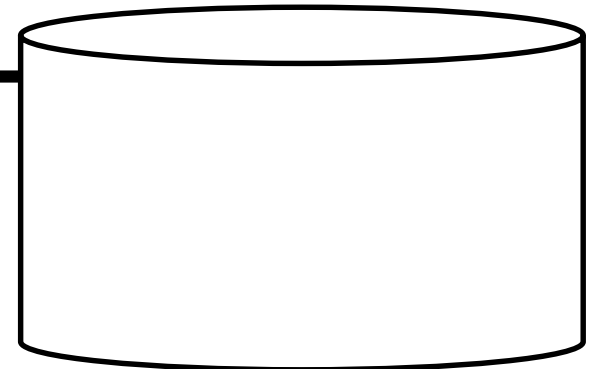
```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    y = 6;
    temp = x;
    x = y;
    y = temp;
    return 0;
}
```



Memória

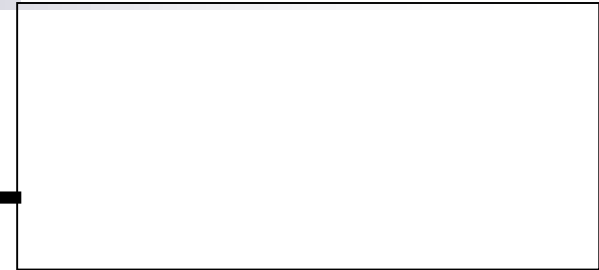
x	y	temp
6	2	2

HD

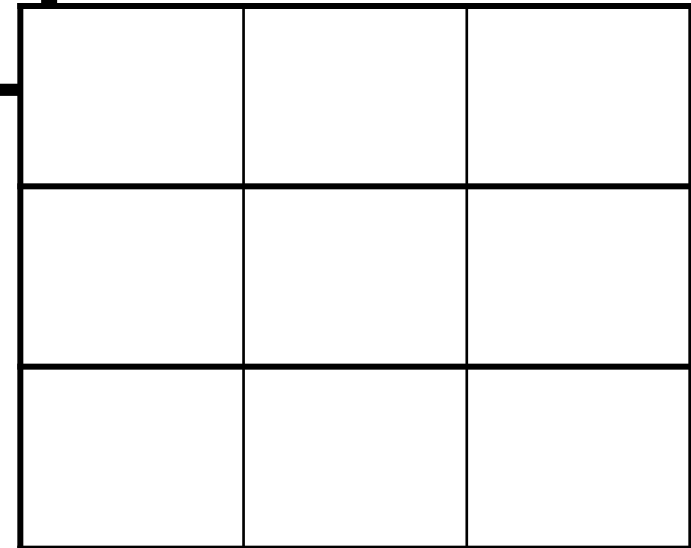


Execução

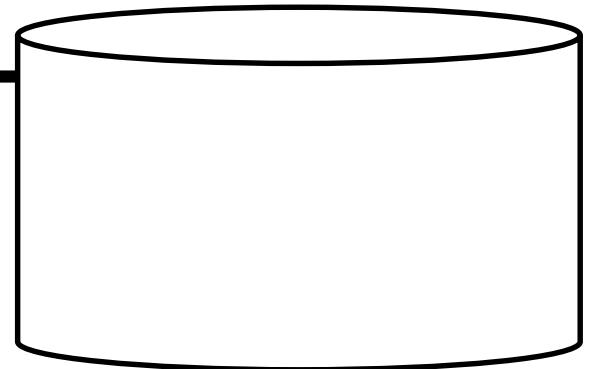
```
#include<stdio.h>
int main() {
    int x; int y; int temp;
    x = 2;
    y = 6;
    temp = x;
    x = y;
    y = temp;
    return 0;
}
```



Memória



HD



Atribuição

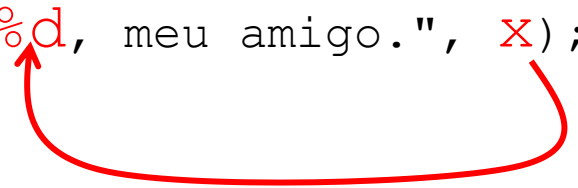
- Que valores x e y possuem ao final da execução?

```
#include<stdio.h>
int main() {
    int x;
    int y;
    x = 5;
    y = 3;
    x = y - x;
    y = 2 * y * (-x) ;
    return 0;
}
```

Atribuição e printf

- Como mostrar o valor de uma variável no monitor?

```
#include <stdio.h>
int main() {
    int x;
    x = 8;
    printf("A variavel tem valor %d, meu amigo.", x);
    return 0;
}
```



Ao invés de mostrar o texto

“A variavel tem valor %d, meu amigo.”

o computador substitui %d pelo valor da variável que aparece após a vírgula (no caso, ele substitui pelo valor da variável x).

Ou seja, aparece

“A variavel tem valor 8, meu amigo.”

Atribuição e printf

■ E agora?

```
#include<stdio.h>
int main() {
    int x;
    int y;
    x = 5;
    y = 3;
    x = y - x;
    y = 2 * y * (-x);
    printf("O valor de x = %d e o de y = %d", x, y);
    return 0;
}
```

O primeiro %d é substituído pelo valor de x e o segundo %d é substituído pelo valor de y.

Isto acontece porque, após as aspas, x vem primeiro e y vem depois.

Atribuição e printf

■ E este? Tem diferença?

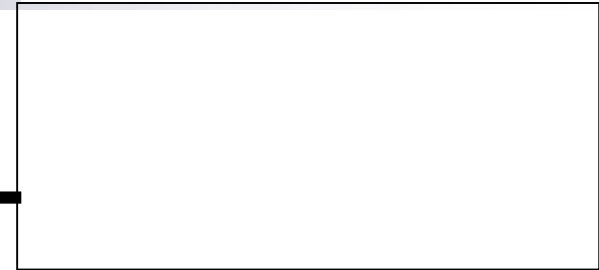
```
#include<stdio.h>
int main() {
    int x;
    int y;
    x = 5;
    y = 3;
    x = y - x;
    y = 2 * y * (-x);
    printf("O valor de x = %d e o de y = %d", y, x);
    return 0;
}
```



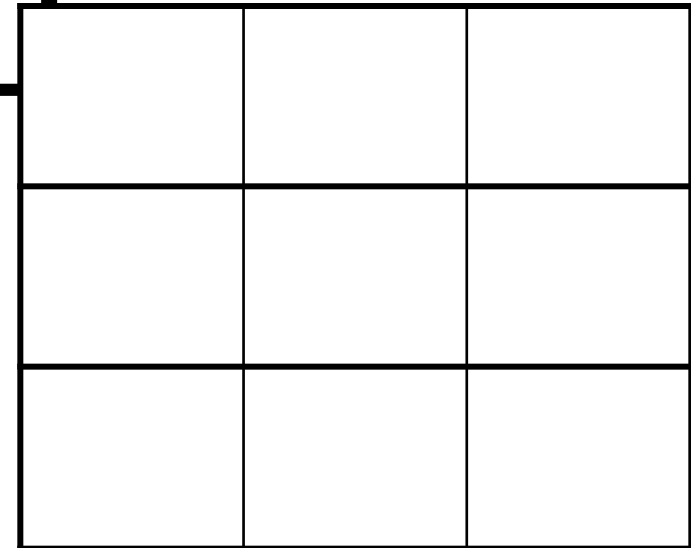
Variável e scanf

Execução

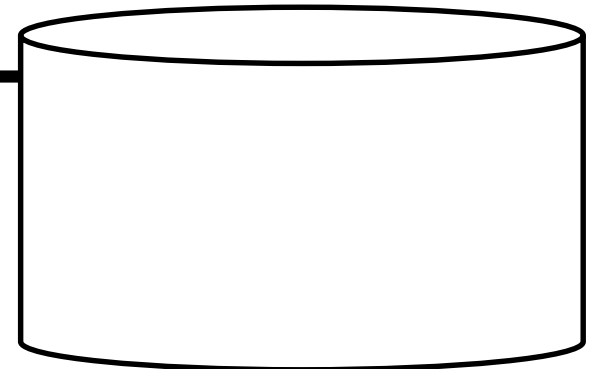
```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```



Memória

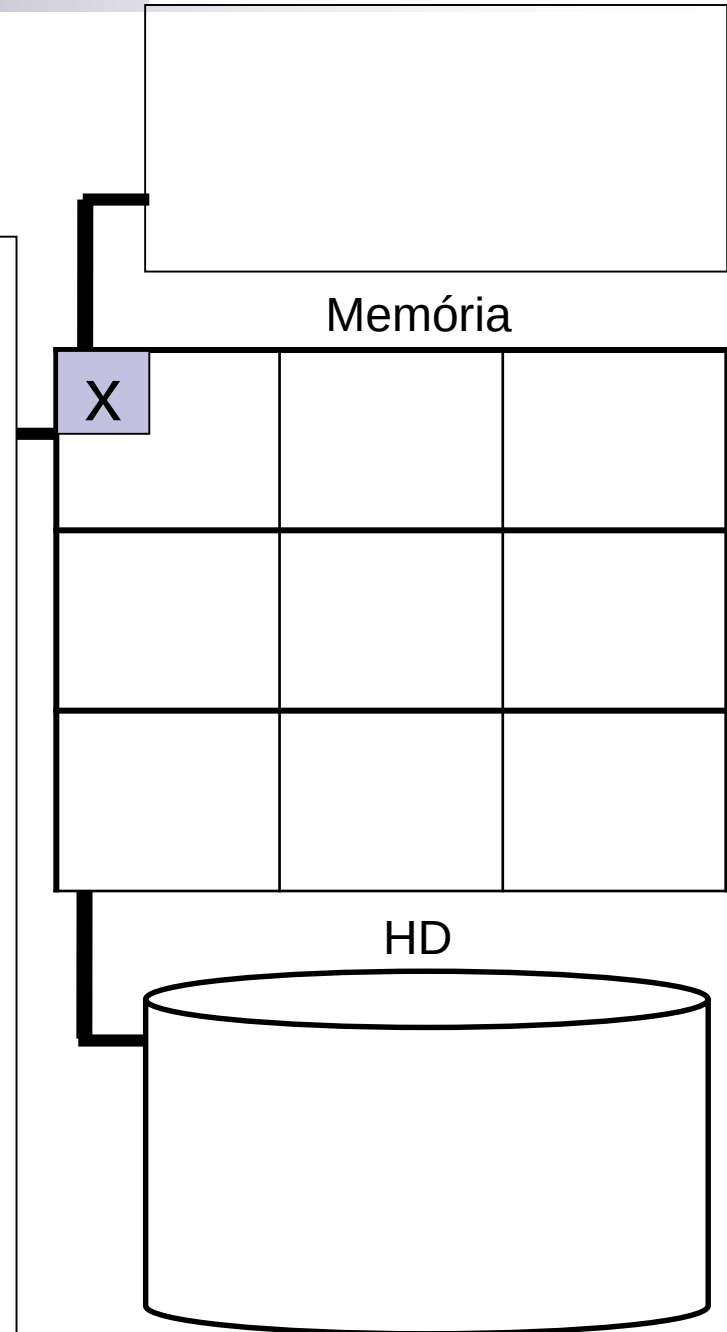


HD



Execução

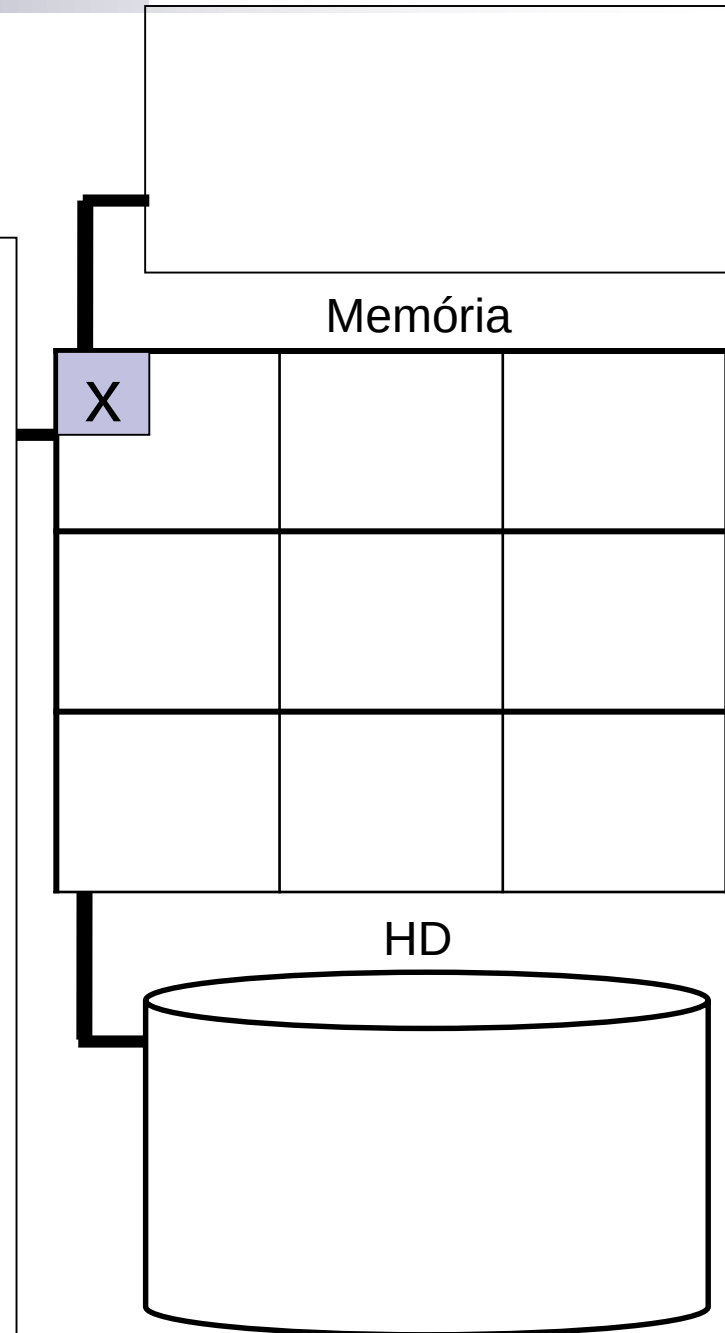
```
#include <stdio.h>
int main() {
    → int x;
    scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```



Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

Neste ponto, o computador espera o usuário digitar um número e teclar ENTER.



Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

Assumimos que o usuário digitou 5.

5

Memória

X

HD

Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

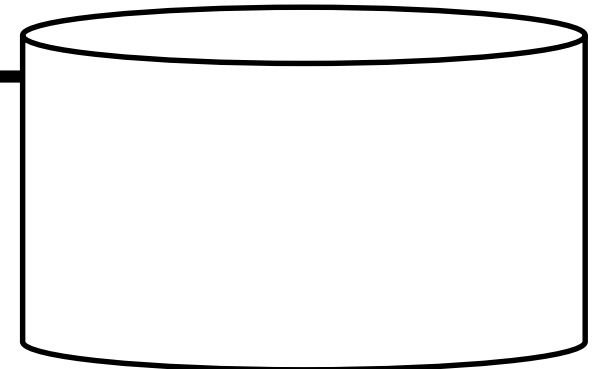
Ao teclar <ENTER>, o computador grava 5 na variável x.

5 <ENTER>

Memória

x		
5		

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    → printf("%d\n", x);
    return 0;
}
```

O printf() pega o valor de x e substitui pelo %d. Ou seja, o texto a ser colocado no monitor é "5\n".

5 <ENTER>

5

Memória

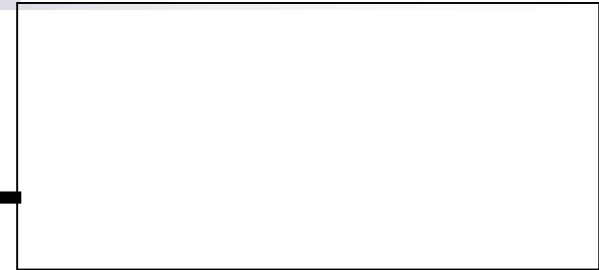
x

5

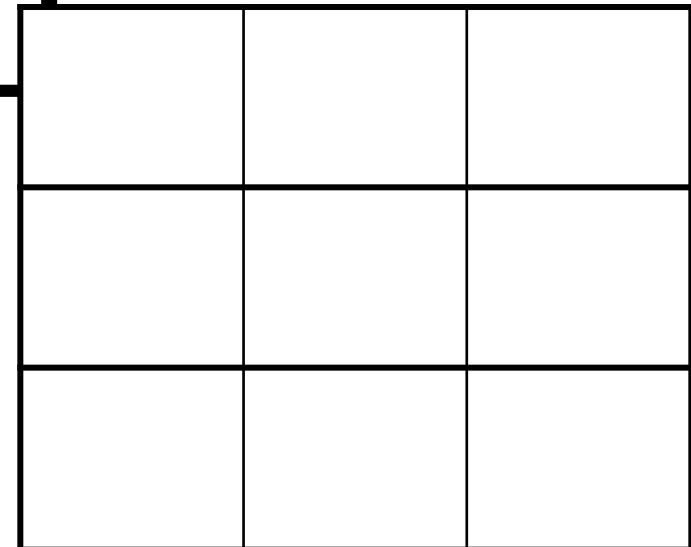
HD

Execução

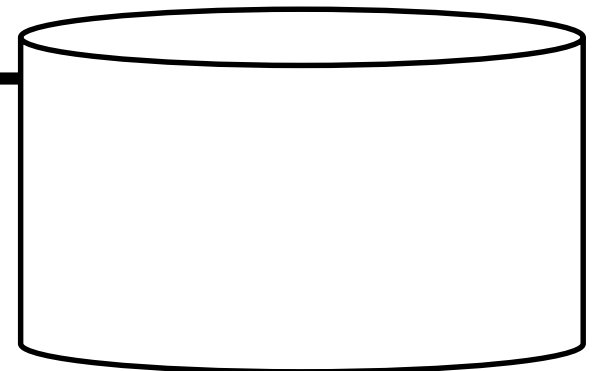
```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    printf("%d\n", x);
    → return 0;
}
```



Memória

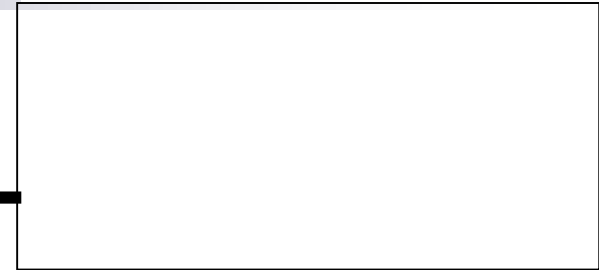


HD

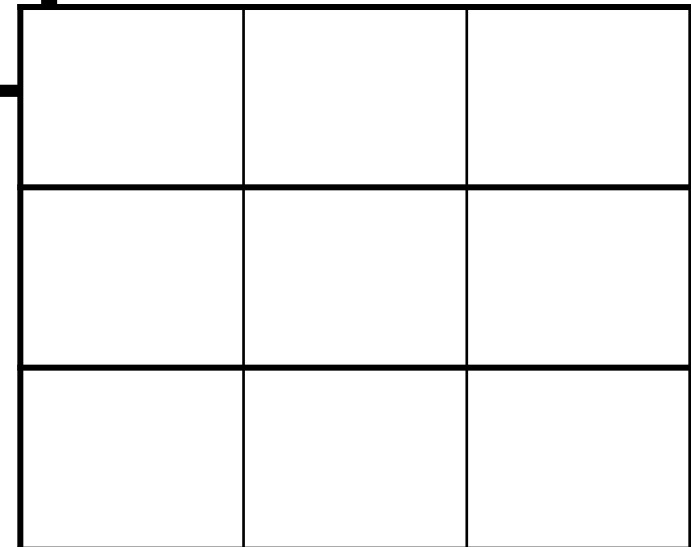


Execução

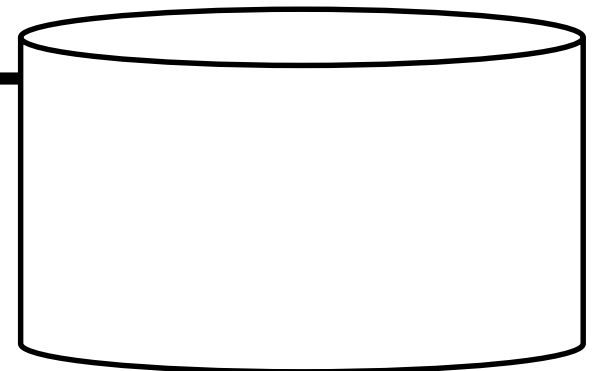
```
#include <stdio.h>
→ int main() {
    int x;
    scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    return 0;
}
```



Memória

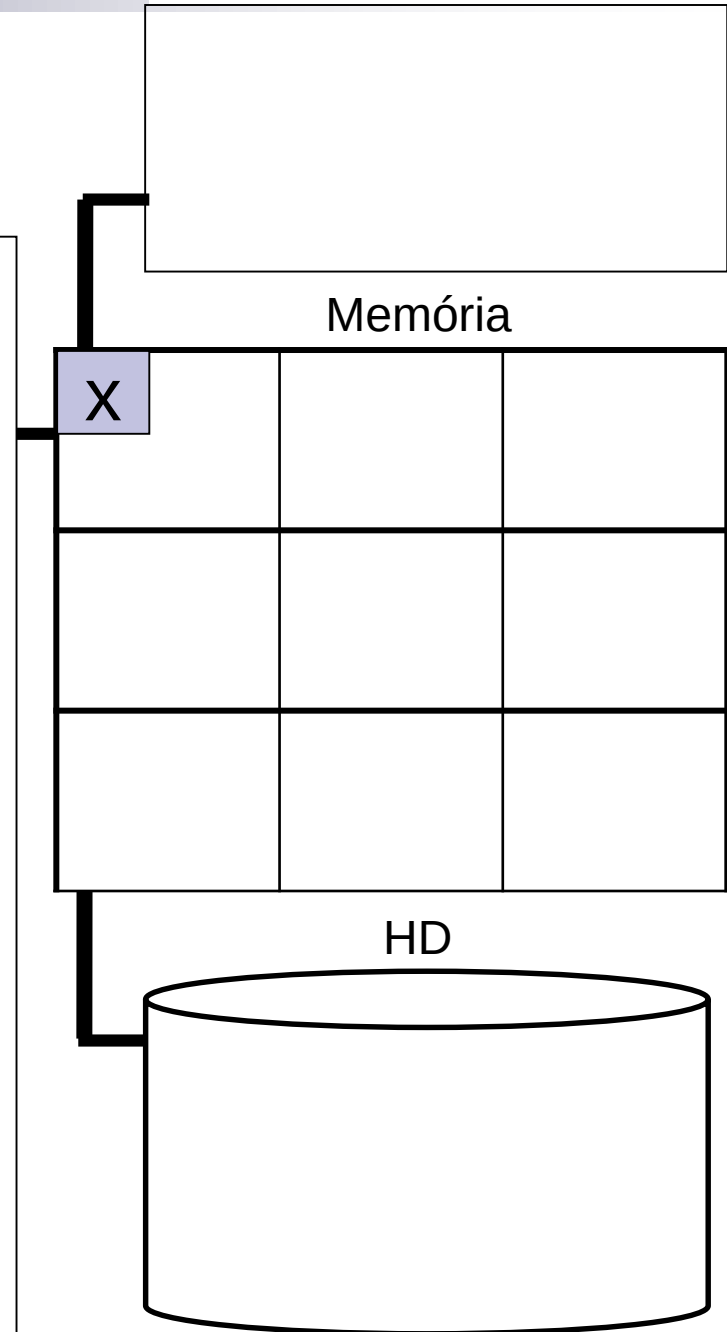


HD



Execução

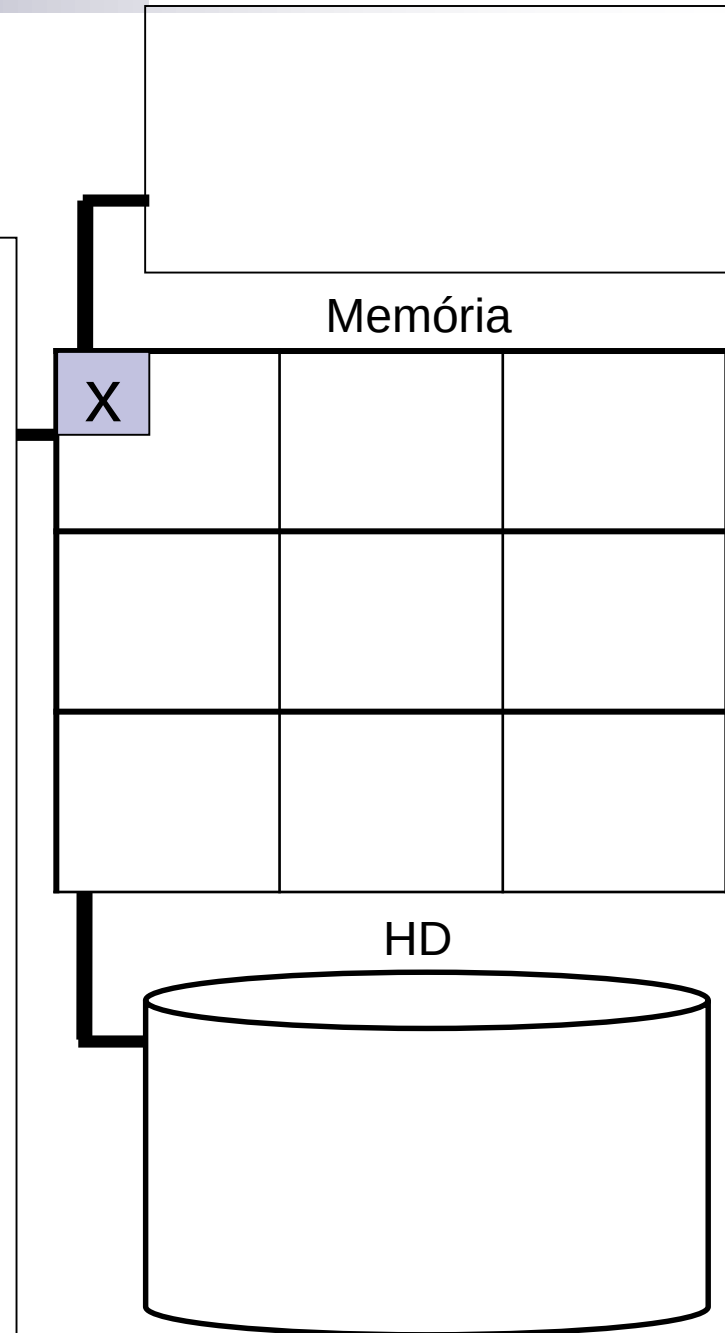
```
#include <stdio.h>
int main() {
    → int x;
    scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    return 0;
}
```



Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    return 0;
}
```

Neste ponto, o computador espera o usuário digitar um número e teclar ENTER.



Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    return 0;
}
```

Vamos assumir que o usuário digita 3.

3

Memória

X

HD

Execução

```
#include <stdio.h>
int main() {
    int x;
    → scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    return 0;
}
```

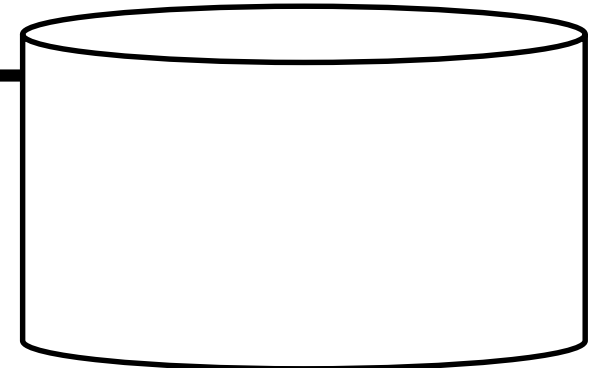
Ao teclar <ENTER>, o 3 vai para x.

3 <ENTER>

Memória

x	3		

HD



3 <ENTER>

Execução

```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    → x = x + x;
    printf("%d\n", x);
    return 0;
}
```

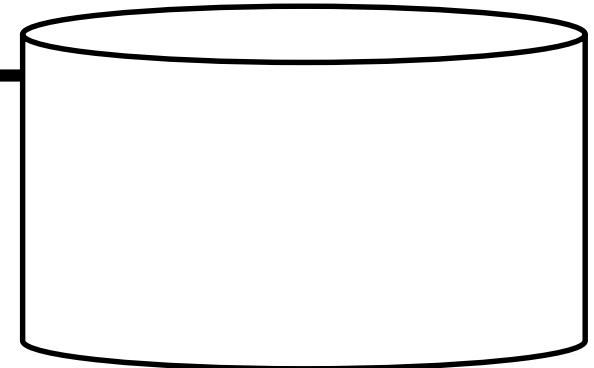
$x + x$
 $= 3 + 3$
 $= 6$

O computador primeiro calcula $x + x$.

Memória

x	3		

HD



3 <ENTER>

Execução

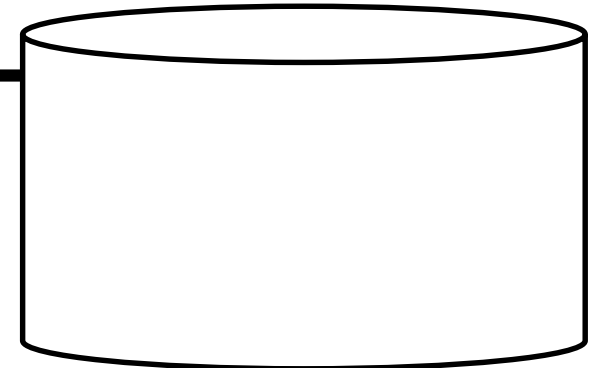
```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    → x = x + x;
    printf("%d\n", x);
    return 0;
}
```

$$\begin{aligned} x + x \\ = 3 + 3 \\ = 6 \end{aligned}$$

Memória

x	6		

HD



Depois, o resultado 6 é gravado em x.

Execução

```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    x = x + x;
    → printf("%d\n", x);
    return 0;
}
```

3 <ENTER>

6

Memória

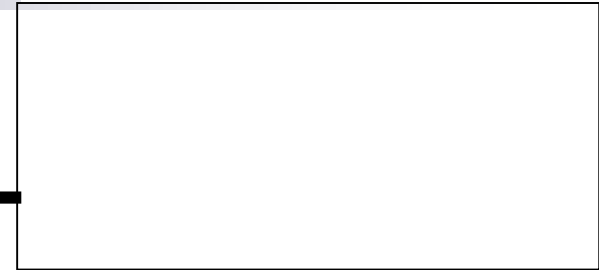
x

6

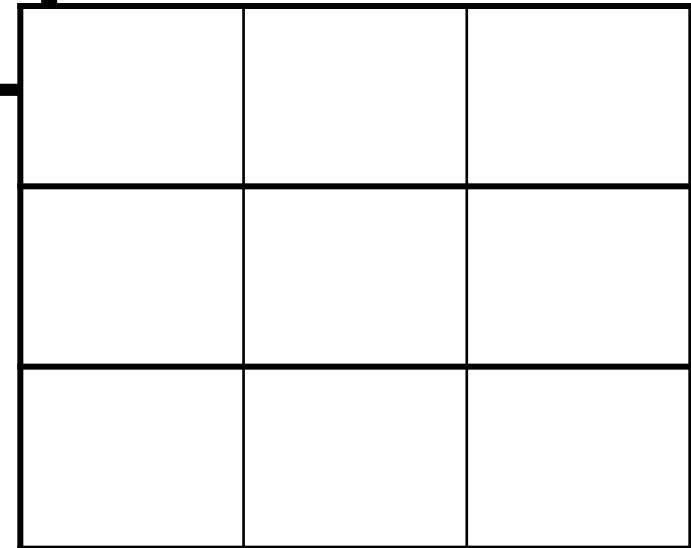
HD

Execução

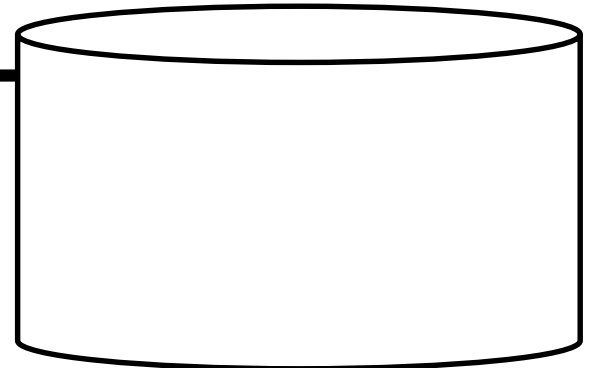
```
#include <stdio.h>
int main() {
    int x;
    scanf("%d", &x);
    x = x + x;
    printf("%d\n", x);
    → return 0;
}
```



Memória



HD



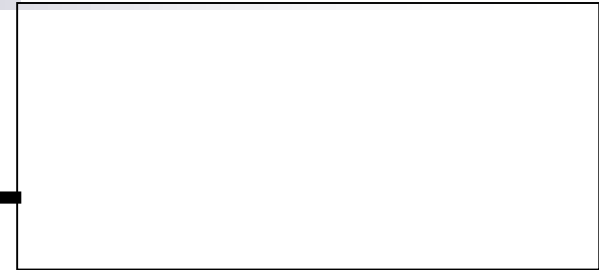
Operadores matemáticos para *inteiros*

- Além de adição (+), subtração (-) e multiplicação (*), C também oferece
 - Divisão: /
 - $15 / 3 = 5$
 - $5 / 2 = 2$ /* Lembre-se: divisão entre inteiros! */
 - Resto ou módulo da divisão: %
 - $15 \% 3 = 0$
 - $5 \% 2 = 1$

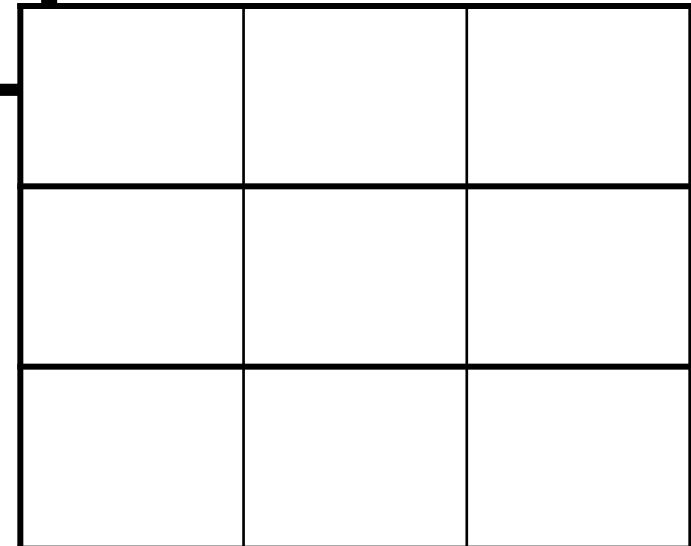
Execução

```
#include <stdio.h>
```

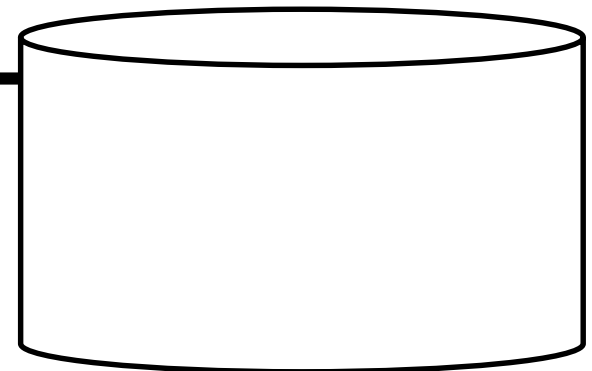
```
→ int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```



Memória



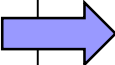
HD



Execução

```
#include <stdio.h>
```

```
int main() {
```



```
int x,y,divisao,resto;
```

```
printf("Entre o numerador: ");
```

```
scanf("%d",&x);
```

```
printf("Entre o denominador: ");
```

```
scanf("%d",&y);
```

```
divisao = x /y;
```

```
resto = x % y;
```

```
printf("Divisao: ");
```

```
printf("%d \n",divisao);
```

```
printf("Resto: ");
```

```
printf("%d \n", resto);
```

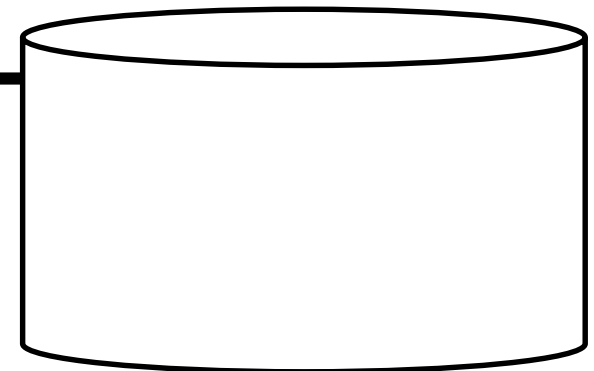
```
return 0;
```

```
}
```

Memória

x	y	divisao
resto		

HD



Execução

```
#include <stdio.h>
```

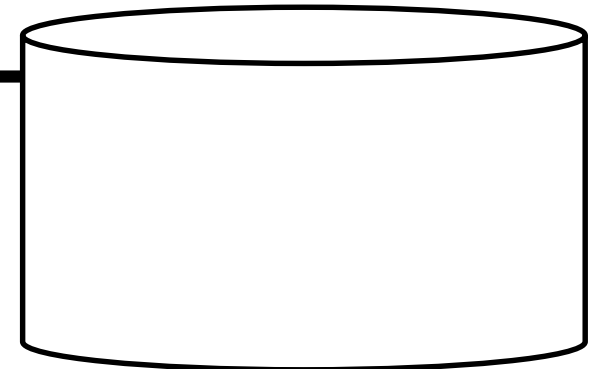
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador:

Memória

x	y	divisao
resto		

HD



Entre o numerador: 7

Execução

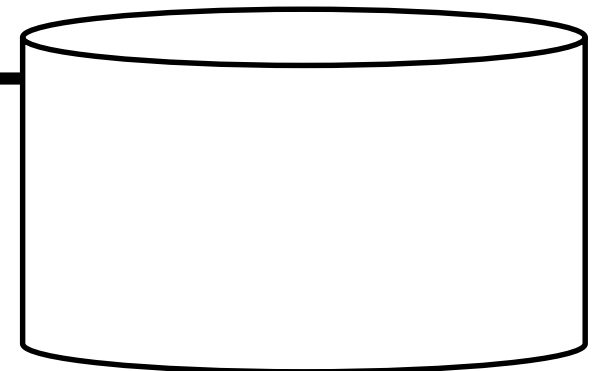
```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Memória

x	y	divisao
resto		

HD



Execução

```
#include <stdio.h>
```

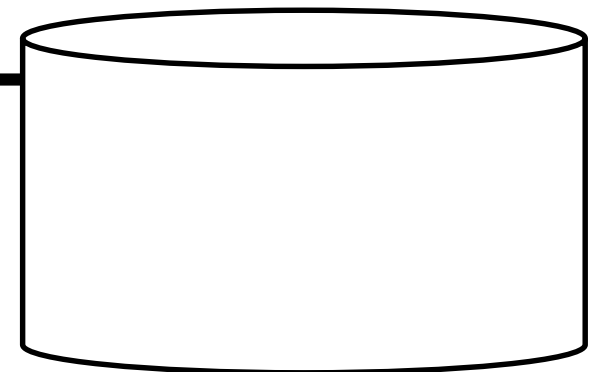
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>

Memória

x	y	divisao
7		
resto		

HD



Execução

```
#include <stdio.h>
```

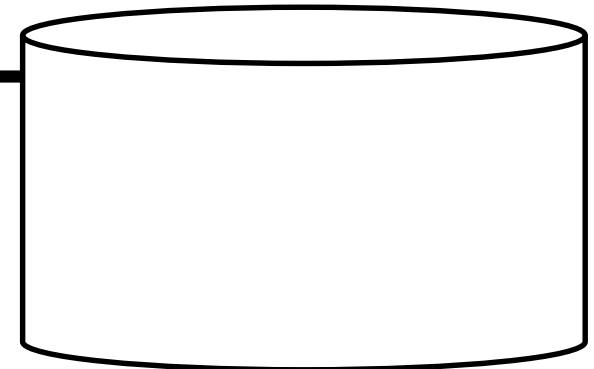
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador:

Memória

x	y	divisao
7		
resto		

HD



Execução

```
#include <stdio.h>
```

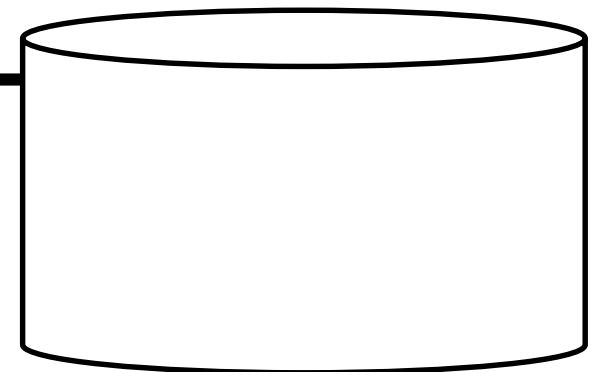
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador: 3

Memória

x	y	divisao
7		
resto		

HD



Execução

```
#include <stdio.h>
```

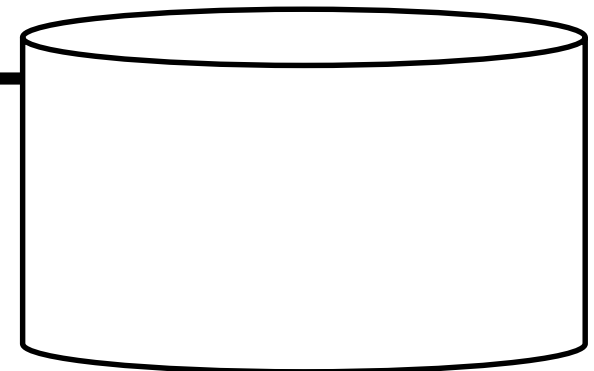
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador: 3
<ENTER>

Memória

x	y	divisao
7		
resto		

HD



Execução

```
#include <stdio.h>
```

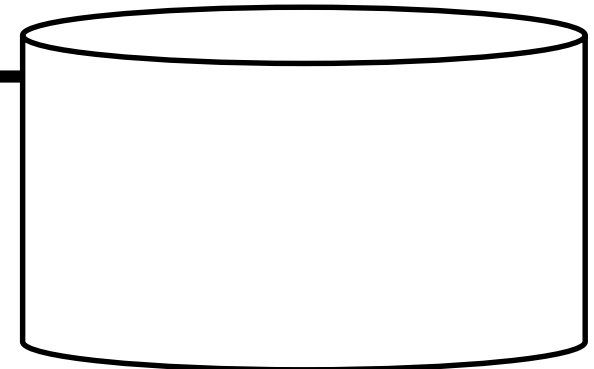
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador: 3
<ENTER>

Memória

x	7	y	3	divisao
resto				

HD



Execução

```
#include <stdio.h>
```

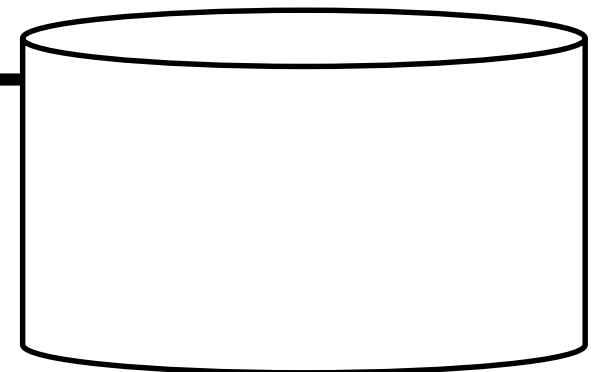
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador: 3
<ENTER>

Memória

x	y	divisao
7	3	2
resto		

HD



Execução

```
#include <stdio.h>
```

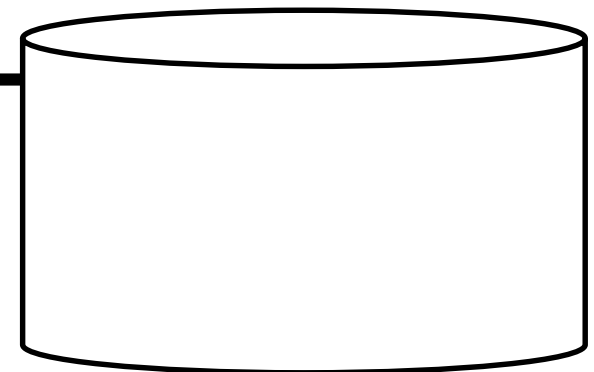
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o numerador: 7
<ENTER>
Entre o denominador: 3
<ENTER>

Memória

x	y	divisao
7	3	2
resto		
1		

HD



Execução

```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

<ENTER>

Entre o denominador: 3

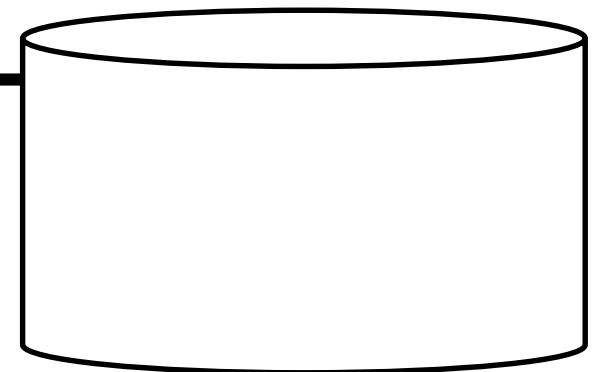
<ENTER>

Divisão:

Memória

x	y	divisao
7	3	2
resto		
1		

HD



Execução

```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

<ENTER>

Entre o denominador: 3

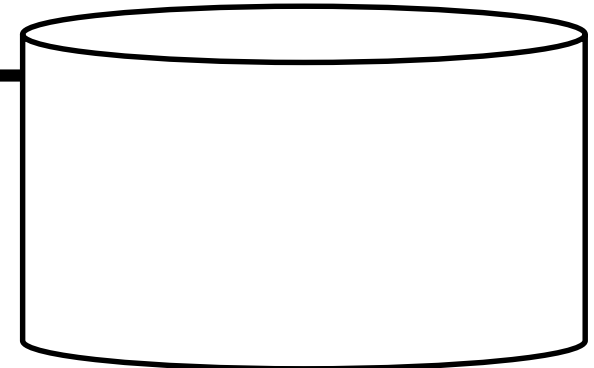
<ENTER>

Divisão: 2

Memória

x	y	divisao
7	3	2
resto		
1		

HD



Execução

```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o denominador: 3

<ENTER>

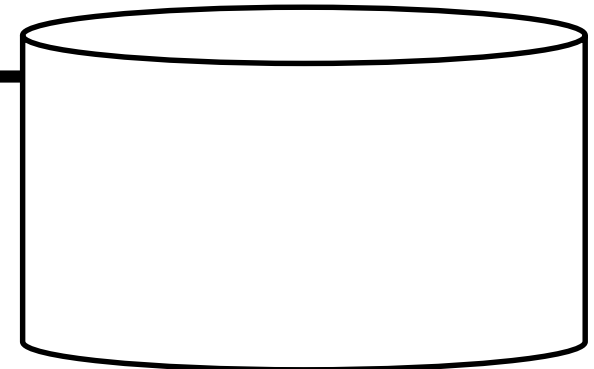
Divisão: 2

Resto:

Memória

x	y	divisao
7	3	2
resto		
1		

HD



Execução

```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```

Entre o denominador: 3

<ENTER>

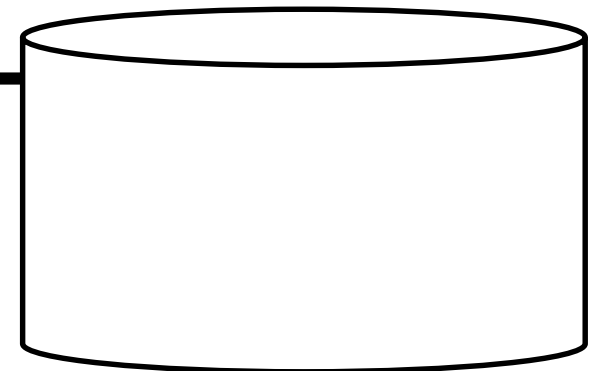
Divisão: 2

Resto: 1

Memória

x	y	divisao
7	3	2
resto		
1		

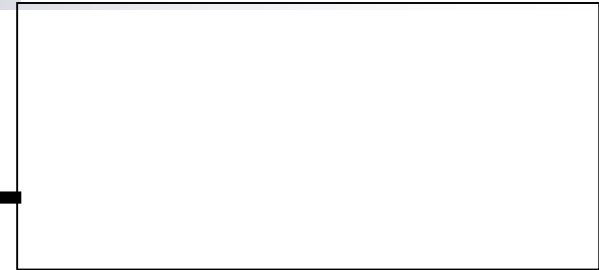
HD



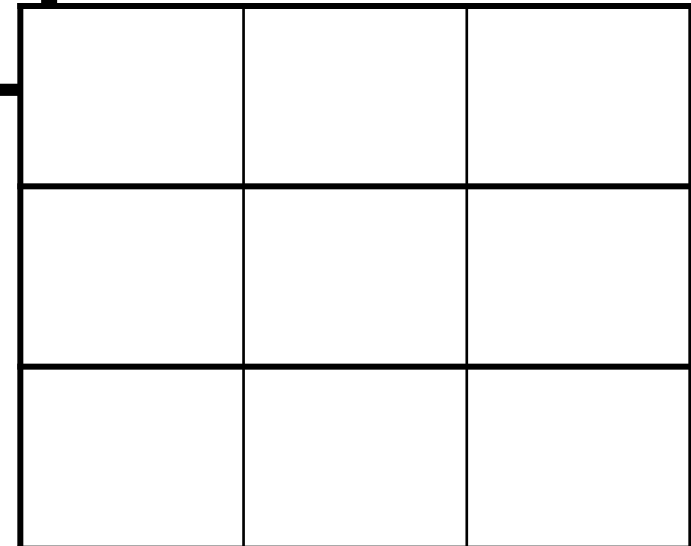
Execução

```
#include <stdio.h>
```

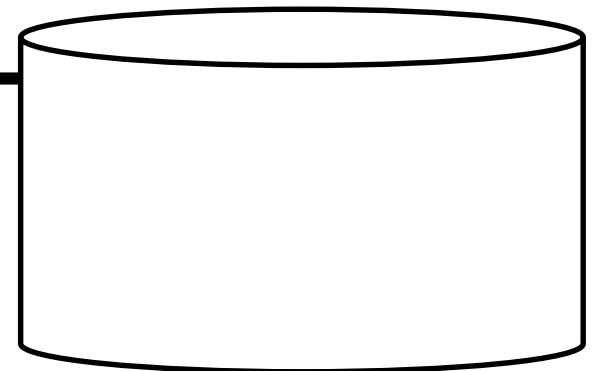
```
int main() {  
    int x,y,divisao,resto;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x /y;  
    resto = x % y;  
    printf("Divisao: ");  
    printf("%d \n",divisao);  
    printf("Resto: ");  
    printf("%d \n", resto);  
    return 0;  
}
```



Memória



HD



Mais sobre printf e scanf

```
#include <stdio.h>
```

```
int main() {  
    int x,y,divisao;  
    printf("Entre o numerador: ");  
    scanf("%d",&x);  
    printf("Entre o denominador: ");  
    scanf("%d",&y);  
    divisao = x / y;  
    printf("Divisao: %d\n",divisao);  
    printf("Resto: %d \n", (x % y));  
  
    return 0;  
}
```

Programa que faz a mesma coisa que o anterior.

Mas não usa tantas variáveis.

Neste caso, o valor resto é calculado dentro do printf.

Mais sobre printf e scanf

- `printf("txtformato", var1, var2, var3,...)`
- comando de saída
 - `txtformato` = o quê e os valores das variáveis a serem impressos
 - Códigos de formatação
 - `\n` = quebra de linha
 - `\t` = Tabulação (espaço de um *tab*)
 - `\a` = Sinal sonoro (*beep*)
 - ... (leia o capítulo 2 do livro texto)
 - `var1` = valor da variável 1;...

Mais sobre printf e scanf

- `scanf("tipo", &var1)` – comando de saída
 - tipo = o tipo da variável
 - %d = inteiro
 - %f = float
 - ... (veja o capítulo 2 do livro texto)
 - &var1 (não esqueça o &)

Exercício

- Fazer um programa para:

- ☐ Ler, via teclado, um número inteiro. Assuma que o usuário vai digitar um número entre 100 e 999.
- ☐ Imprimir no monitor os dígitos deste número (1 dígito em cada linha).

- Exemplo. Se o usuário digitar 358, imprimir:

3

5

8

Exercício

- Fazer um programa para:
 - Ler, via teclado, 3 números inteiros x , y e z . Assuma que o usuário vai digitar números entre 100 e 999.
 - Imprimir no monitor a soma dos dígitos destes números.
 - Exemplo. Se o usuário digitar 353, 612 e 999 para x , y e z respectivamente, o programa deve imprimir 11, 9 e 27.